



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	T5. Estructuras de control
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Ashanga Yumbo Brishy Anahy Calderón Carvajal Fher Dorian Casillas Ochoa Antony Sebastián Paredes Acosta Dereck Shair Sánchez Bastidas Karina Paola
Asignatura:	Algoritmos y Lógica de programación
Docente:	Ing. José Ruben Caizabuano, M.Sc.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar soluciones algorítmicas en Java integrando estructuras for, while y do-while, ciclos anidados, contadores, acumuladores, validación de datos, casos límite y depuración.

Específicos:

- Aplicar estructuras de control en Java, como for, while, do-while y ciclos anidados, para resolver diferentes problemas algorítmicos.
- Implementar técnicas de control y procesamiento de datos, mediante contadores, acumuladores y validaciones, considerando distintos casos límite durante la ejecución de los programas.
- Documentar y evidenciar el proceso de desarrollo, prueba y depuración de las soluciones algorítmicas mediante un repositorio en GitHub, manteniendo organizado el código y los avances realizados.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 2

No presenciales: 0

2.4 Listado de equipos, materiales y recursos

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☒ Gamificación
- ☒ Inteligencia Artificial
- Otros (Especifique): _____

2.5 Actividades por desarrollar



Resolver diez ejercicios de programación en Java, aplicando estructuras de control for, while y do-while, ciclos anidados, contadores, acumuladores y validación de datos. Cada ejercicio deberá incluir el análisis del problema, desarrollo del algoritmo, codificación, pruebas de funcionamiento, identificación de casos límite y depuración de errores, evidenciando todo el proceso mediante un repositorio en GitHub.

2.6 Resultados obtenidos

PRUEBA DE ESCRITOTIO

Resumen de los casos Ejercicio uno - CONTROL DE CALIFICACIONES

Caso	N	Suma	Promedio	Aprobados	Reprobados	Mayor / menor
1	3	17,00	5,67	2	1	10,00 / 0,00
2	4	31,00	7,75	3	1	9,50 / 6,50
3	2	12,00	6,00	1	1	7,00 / 5,00

Resumen de los casos Ejercicio dos – TABLA DE MULTIPLICAR CONFIGURABLE

Caso	Intervalo	Multiplicador final	Tablas	Operaciones	Último resultado
1	3 a 5	10	3	30	$5 \times 10 = 50$
2	6 a 4 $\rightarrow$ 2 a 4	5	3	15	$4 \times 5 = 20$
3	7 a 7	0 $\rightarrow$ 12	1	12	$7 \times 12 = 84$

Resumen de los casos Ejercicio tres – SERIE Y SUMA DE NÚMEROS PARES

Caso	N ingresado	N válido	Serie	Suma	Promedio
1	12	12	2, 4, 6, 8, 10, 12	42	7,00
2	1	1	Sin pares	0	No se calcula
3	-5	2	2	2	2,00

Resumen de los casos Ejercicio cuatro - CAJERO AUTOMÁTICO BÁSICO

Caso	Depósitos	Retiros	Total depositado	Total retirado	Saldo final
1	1	1	\$50,00	\$30,00	\$120,00
2	0	0	\$0,00	\$0,00	\$100,00
3	1	0	\$25,00	\$0,00	\$125,00



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



Resumen de los casos Ejercicio cinco - ESTACIONAMIENTO UNIVERSITARIO

Caso	Motos	Autos	Camionetas	Total	Recaudación	Promedio
1	1	0	0	1	\$2,00	\$2,00
2	0	1	1	2	\$6,00	\$3,00
3	1	1	1	3	\$7,00	\$2,33

Resumen de los casos Ejercicio seis - TRIÁNGULO Y PATRONES

Caso	Entrada	N válido	Filas por patrón	Repeticiones internas por patrón
1	5	5	5	15
2	2	2	2	3
3	11 → 10	10	10	55

Resumen de los casos Ejercicio siete - CONTROL DE VENTAS DE CAFETERÍA

Caso	Ventas	Unidades	Total	Promedio	Líder
1	0	0	\$0.00	No aplica	Ninguno
2	4	10	\$14.00	\$3.50	Empanada
3	2	4	\$5.00	\$2.50	Café y Jugo

Resumen de los casos Ejercicio ocho - DETECTOR Y DEPURADOR DE ERRORES

Caso	Comportamiento comprobado	Resultado
1. Original	Falta la actualización	Repite 1 indefinidamente
2. Corregido	Incluye 10 y sale en 11	10 impresiones
3. Adicional corregido	Incluye 1 y sale en 0	5 impresiones

Resumen de los casos Ejercicio nueve - ESTADÍSTICAS DE UNA ENCUESTA UNIVERSITARIA

Caso	Edad prom.	Horas prom.	Máximo	Menos de 2 h
1	20,00	2,50	E2: 4 h	1
2	48,00	12,00	E2: 24 h	1
3	19,00	2,00	E1: 2 h	0

Resumen de los casos Ejercicio diez - SISTEMA INTEGRADOR DE PARQUEADERO

Caso	Registros	Horas / prom.	Recaudación	Mayor / menor
1. Vacío	0	0 / no disponible	\$0,00	No disponibles
2. Variados	3	6 / 2,00	\$11,39	\$8,24 / \$0,75
3. Validaciones	1	24 / 24,00	\$23,00	\$23,00 / \$23,00



2.7 Habilidades blandas empleadas en la práctica

- ☒ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☒ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.8 Conclusiones

- La resolución de los diez ejercicios permitió fortalecer el uso de estructuras de control como `for`, `while` y `do-while`, facilitando la construcción de soluciones algorítmicas ordenadas y funcionales.
- La aplicación de contadores, acumuladores, ciclos anidados y validaciones permitió desarrollar programas más completos, capaces de procesar correctamente diferentes tipos de datos y situaciones.
- El uso de GitHub permitió organizar y evidenciar el proceso de desarrollo de los ejercicios, facilitando el seguimiento de los avances y la documentación de las soluciones realizadas.

2.9 Recomendaciones

- Realizar pruebas con diferentes valores de entrada, incluyendo casos límite, para comprobar que cada programa funcione correctamente ante distintas situaciones.
- Mantener el código organizado, utilizando nombres de variables claros, indentación adecuada y comentarios únicamente cuando sean necesarios para facilitar su comprensión.
- Utilizar GitHub de manera constante durante el desarrollo de los ejercicios, realizando actualizaciones periódicas que permitan conservar un historial ordenado de los cambios y mejoras realizadas.

2.10 Anexos

Link del GitHub: <https://github.com/Doros-ia/Ejercicios-deber/tree/main>

CONTROL DE CALIFICACIONES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrollar un programa que permita ingresar las calificaciones de N estudiantes. El número de estudiantes debe ser mayor que cero y cada calificación deberá estar entre 0 y 10. El programa determinará el número de estudiantes, la suma de calificaciones, el promedio general, la cantidad de aprobados, la cantidad de reprobados, la nota más alta y la nota más baja. Se utilizará while para las validaciones y for para el procesamiento. Como el enunciado incluye la nota 7 como caso límite, se considera aprobado al estudiante que obtiene una calificación mayor o igual que 7.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Número de estudiantes N: dato entero mayor que cero.
- Calificación de cada estudiante: dato real entre 0 y 10, incluidos ambos límites.

1.2 Proceso

- Validar con while que el número de estudiantes sea mayor que cero.
- Inicializar la suma, el contador de aprobados y el contador de reprobados en cero.
- Recorrer los N estudiantes mediante un ciclo for.
- Solicitar y validar con while cada calificación entre 0 y 10.
- Acumular cada calificación en la suma total.
- Usar la primera calificación para inicializar la nota mayor y la nota menor.
- Comparar las calificaciones siguientes para actualizar la nota mayor y la nota menor.
- Incrementar aprobados si la nota es mayor o igual que 7; en caso contrario, incrementar reprobados.
- Calcular promedio = suma / número de estudiantes. La división es segura porque N fue validado como mayor que cero.

1.3 Salida de datos

- Número de estudiantes procesados.
- Suma de las calificaciones.
- Promedio general.
- Cantidad de estudiantes aprobados.
- Cantidad de estudiantes reprobados.
- Nota más alta.
- Nota más baja.
- Mensajes de validación para cantidades o calificaciones incorrectas.

2. ALGORITMO

1. **Inicio.**
2. Declarar las variables para el número de estudiantes, calificación, suma, promedio, aprobados, reprobados, nota mayor y nota menor.
3. Solicitar el número de estudiantes.
4. Mientras el número sea menor o igual que cero, mostrar un mensaje y solicitarlo nuevamente.
5. Inicializar suma, aprobados y reprobados en cero.
6. Repetir desde el estudiante 1 hasta el estudiante N mediante un ciclo for.
7. Solicitar la calificación del estudiante.
8. Mientras la calificación sea menor que 0 o mayor que 10, mostrar un mensaje y solicitarla nuevamente.
9. Agregar la calificación a la suma total.
10. Si es la primera calificación, asignarla como nota mayor y nota menor.
11. En las demás iteraciones, actualizar la nota mayor o menor cuando corresponda.
12. Si la calificación es mayor o igual que 7, incrementar aprobados; si no, incrementar reprobados.
13. Al terminar el ciclo, calcular el promedio dividiendo la suma para N.
14. Mostrar el número de estudiantes, suma, promedio, aprobados, reprobados, nota mayor y nota menor.
15. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo ControlDeCalificaciones

```
Definir N, i, aprobados, reprobados Como Entero;
Definir calificacion, suma, promedio, notaMayor, notaMenor Como Real;
Escribir "Ingrese el número de estudiantes:";
Leer N;
Mientras N <= 0 Hacer
    Escribir "La cantidad debe ser mayor que cero";
    Escribir "Ingrese nuevamente:";
    Leer N;
FinMientras;
suma <- 0;
aprobados <- 0;
reprobados <- 0;
Para i <- 1 Hasta N Con Paso 1 Hacer
    Escribir "Calificación del estudiante ", i, " (0 a 10):";
    Leer calificacion;
    Mientras calificacion < 0 O calificacion > 10 Hacer
        Escribir "Calificación inválida. Debe estar entre 0 y 10";
        Escribir "Ingrese nuevamente:";
        Leer calificacion;
    FinMientras;
    suma <- suma + calificacion;
    Si i = 1 Entonces
        notaMayor <- calificacion;
        notaMenor <- calificacion;
    SiNo
        Si calificacion > notaMayor Entonces
            notaMayor <- calificacion;
        FinSi;
        Si calificacion < notaMenor Entonces
            notaMenor <- calificacion;
        FinSi;
    FinSi;
    Si calificacion >= 7 Entonces
        aprobados <- aprobados + 1;
    SiNo
        reprobados <- reprobados + 1;
    FinSi;
FinPara;
promedio <- suma / N;
Escribir "Número de estudiantes: ", N;
Escribir "Suma de calificaciones: ", suma;
Escribir "Promedio general: ", promedio;
Escribir "Cantidad de aprobados: ", aprobados;
Escribir "Cantidad de reprobados: ", reprobados;
Escribir "Nota más alta: ", notaMayor;
Escribir "Nota más baja: ", notaMenor;
```

FinAlgoritmo

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Límites obligatorios 0, 7 y 10

Variable	Valor
N	3
calificaciones	0; 7; 10
suma	17,00
promedio	5,67
aprobados	2
reprobados	1
nota mayor	10,00
nota menor	0,00

Operaciones:

Estudiante 1: suma = $0 + 0 = 0$; reprobados = 1; mayor = 0; menor = 0.

Estudiante 2: suma = $0 + 7 = 7$; aprobados = 1; mayor = 7; menor = 0.

Estudiante 3: suma = $7 + 10 = 17$; aprobados = 2; mayor = 10; menor = 0.

promedio = $17 / 3 = 5,666... \approx 5,67$.

Caso 2: Calificaciones variadas

Variable	Valor
N	4
calificaciones	6,5; 7; 8; 9,5
suma	31,00
promedio	7,75
aprobados	3
reprobados	1
nota mayor	9,50
nota menor	6,50

Operaciones:

suma = $6,5 + 7 + 8 + 9,5 = 31,00$.

promedio = $31 / 4 = 7,75$.

Notas aprobadas: 7; 8; 9,5. Total = 3.

Nota reprobada: 6,5. Total = 1.

Caso 3: Validación de N y de calificaciones

Variable	Valor
primer N ingresado	0 (inválido)
N corregido	2
primera nota intentada	-1 (inválida)

Variable	Valor
primera nota válida	5
segunda nota intentada	11 (inválida)
segunda nota válida	7
suma	12,00
promedio	6,00
aprobados / reprobados	1 / 1
nota mayor / menor	7,00 / 5,00

Operaciones:

$N = 0$ no cumple $N > 0$; se solicita nuevamente y se acepta $N = 2$.

-1 no pertenece al intervalo $[0, 10]$; se reemplaza por 5.

11 no pertenece al intervalo $[0, 10]$; se reemplaza por 7.

suma = $5 + 7 = 12$; promedio = $12 / 2 = 6,00$.

Resumen de los casos

Caso	N	Suma	Promedio	Aprobados	Reprobados	Mayor / menor
1	3	17,00	5,67	2	1	10,00 / 0,00
2	4	31,00	7,75	3	1	9,50 / 6,50
3	2	12,00	6,00	1	1	7,00 / 5,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;
public class ControlCalificacionesN {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        // Variables simples para registrar y resumir las calificaciones.
        int numeroEstudiantes, aprobados = 0, reprobados = 0;
        double calificacion, suma = 0, promedio;
        double notaMayor = 0, notaMenor = 0;
        // Se valida que la cantidad de estudiantes sea mayor que cero.
        System.out.println("Ingrese el número de estudiantes: ");
        numeroEstudiantes = entrada.nextInt();
        while (numeroEstudiantes <= 0) {
            System.out.println("La cantidad debe ser mayor que cero.");
            System.out.print("Ingrese nuevamente: ");
            numeroEstudiantes = entrada.nextInt();
        }
        // El ciclo for procesa exactamente una nota por estudiante.
        for (int i = 1; i <= numeroEstudiantes; i++) {
            System.out.print("Calificación del estudiante " + i + " (0 a 10): ");
            calificacion = entrada.nextDouble();
            while (calificacion < 0 || calificacion > 10) {
                System.out.println("Calificación inválida. Debe estar entre 0 y 10.");
                System.out.print("Ingrese nuevamente: ");
                calificacion = entrada.nextDouble();
            }
            suma += calificacion;
            // La primera nota sirve para iniciar el valor mayor y el menor.
            if (i == 1) {
                notaMayor = calificacion;
                notaMenor = calificacion;
            } else {
                if (calificacion > notaMayor) {
                    notaMayor = calificacion;
                }
                if (calificacion < notaMenor) {
                    notaMenor = calificacion;
                }
            }
            // Una nota de 7 o más se considera aprobada.
            if (calificacion >= 7) {
                aprobados++;
            } else {
                reprobados++;
            }
        }
        promedio = suma / numeroEstudiantes;
        System.out.println("\n===== RESUMEN DE CALIFICACIONES =====");
        System.out.println("Número de estudiantes: " + numeroEstudiantes);
        System.out.printf("Suma de calificaciones: %.2f\n", suma);
        System.out.printf("Promedio general: %.2f\n", promedio);
        System.out.println("Cantidad de aprobados: " + aprobados);
        System.out.println("Cantidad de reprobados: " + reprobados);
        System.out.printf("Nota más alta: %.2f\n", notaMayor);
        System.out.printf("Nota más baja: %.2f\n", notaMenor);
        entrada.close();
    }
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    // Variables simples para registrar y resumir las
    calificaciones.
    int numeroEstudiantes, aprobados = 0, reprobados = 0;
    double calificacion, suma = 0, promedio;
    double notaMayor = 0, notaMenor = 0;
    // Se valida que la cantidad de estudiantes sea mayor que cero.
    cout << "Ingrese el número de estudiantes: ";
    cin >> numeroEstudiantes;
    while (numeroEstudiantes <= 0) {
        cout << "La cantidad debe ser mayor que cero.\n";
        cout << "Ingrese nuevamente: ";
        cin >> numeroEstudiantes;
    }
    // El ciclo for procesa exactamente una nota por estudiante.
    for (int i = 1; i <= numeroEstudiantes; i++) {
        cout << "Calificación del estudiante " << i << " (0 a 10):
";
        cin >> calificacion;
        while (calificacion < 0 || calificacion > 10) {
            cout << "Calificación inválida. Debe estar entre 0 y
10.\n";
            cout << "Ingrese nuevamente: ";
            cin >> calificacion;
        }
        suma += calificacion;
        // La primera nota sirve para iniciar el valor mayor y el
        menor.
        if (i == 1) {
            notaMayor = calificacion;
            notaMenor = calificacion;
        } else {
            if (calificacion > notaMayor) {
                notaMayor = calificacion;
            }
            if (calificacion < notaMenor) {
                notaMenor = calificacion;
            }
        }
        // Una nota de 7 o más se considera aprobada.
        if (calificacion >= 7) {
            aprobados++;
        } else {
            reprobados++;
        }
    }
    promedio = suma / numeroEstudiantes;
    cout << fixed << setprecision(2);
    cout << "\n==== RESUMEN DE CALIFICACIONES =====\n";
    cout << "Número de estudiantes: " << numeroEstudiantes << '\n';
    cout << "Suma de calificaciones: " << suma << '\n';
    cout << "Promedio general: " << promedio << '\n';
    cout << "Cantidad de aprobados: " << aprobados << '\n';
    cout << "Cantidad de reprobados: " << reprobados << '\n';
    cout << "Nota más alta: " << notaMayor << '\n';
    cout << "Nota más baja: " << notaMenor << '\n';
    return 0;
}
```

TABLA DE MULTIPLICAR CONFIGURABLE

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicitar dos números: una tabla inicial y una tabla final. Generar todas las tablas de multiplicar comprendidas entre esos dos valores. Se debe validar que la tabla inicial no sea mayor que la tabla final y utilizar ciclos for anidados. Como desafío, el usuario también determinará hasta qué multiplicador desea generar cada tabla.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Tabla inicial: número entero desde el cual comienza la generación.
- Tabla final: número entero hasta el cual se generan las tablas; debe ser mayor o igual que la tabla inicial.
- Multiplicador final: número entero mayor que cero que indica hasta qué operación se mostrará cada tabla.

1.2 Proceso

- Leer la tabla inicial y la tabla final.
- Validar con while que la tabla inicial no sea mayor que la tabla final; si no se cumple, volver a solicitar ambos valores.
- Leer el multiplicador final y validar que sea mayor que cero.
- Usar un ciclo for externo para recorrer cada tabla desde la inicial hasta la final.
- Para cada tabla, usar un ciclo for interno desde 1 hasta el multiplicador final.
- En cada repetición calcular resultado = tabla $\times$ multiplicador y mostrar la operación.

1.3 Salida de datos

- Mensaje de error cuando el intervalo de tablas no es válido.
- Mensaje de error cuando el multiplicador final no es positivo.
- Título de cada tabla generada.
- Operaciones y resultados desde tabla $\times$ 1 hasta tabla $\times$ multiplicador final.

2. ALGORITMO

1. **Inicio.**
2. Declarar tablaInicial, tablaFinal, multiplicadorFinal, tabla, multiplicador y resultado como enteros.
3. Solicitar la tabla inicial y la tabla final.
4. Mientras tablaInicial sea mayor que tablaFinal, mostrar un error y solicitar nuevamente ambos valores.
5. Solicitar el multiplicador final.
6. Mientras el multiplicador final sea menor o igual que cero, mostrar un error y solicitarlo nuevamente.

7. Recorrer con un ciclo for cada tabla desde tablaInicial hasta tablaFinal.
8. Mostrar el título de la tabla actual.
9. Dentro del ciclo anterior, recorrer con otro for los multiplicadores desde 1 hasta multiplicadorFinal.
10. Calcular resultado = tabla × multiplicador.
11. Mostrar la tabla, el multiplicador y el resultado.
12. Al finalizar el ciclo interno, avanzar a la tabla siguiente.
13. **Fin.**

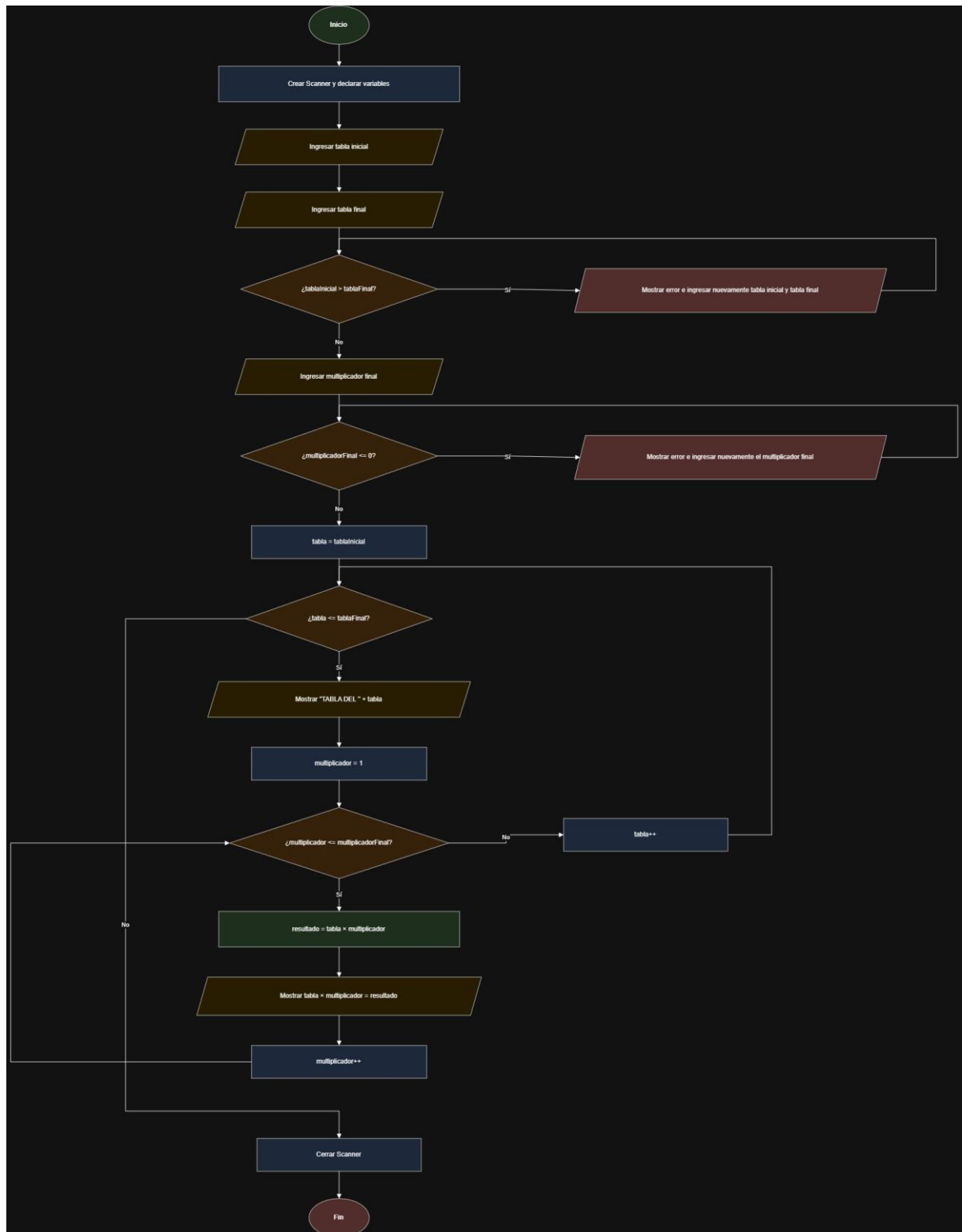
3. PSEUDOCÓDIGO

Algoritmo TablaMultiplicarConfigurable

```
Definir tablaInicial, tablaFinal, multiplicadorFinal Como Entero;
Definir tabla, multiplicador, resultado Como Entero;
Escribir "Ingrese la tabla inicial:";
Leer tablaInicial;
Escribir "Ingrese la tabla final:";
Leer tablaFinal;
Mientras tablaInicial > tablaFinal Hacer
    Escribir "Error: la tabla inicial no puede ser mayor que la tabla final";
    Escribir "Ingrese nuevamente la tabla inicial:";
    Leer tablaInicial;
    Escribir "Ingrese nuevamente la tabla final:";
    Leer tablaFinal;
FinMientras;
Escribir "Ingrese el multiplicador final, mayor que 0:";
Leer multiplicadorFinal;
Mientras multiplicadorFinal <= 0 Hacer
    Escribir "Error: el multiplicador final debe ser mayor que 0";
    Escribir "Ingrese nuevamente el multiplicador final:";
    Leer multiplicadorFinal;
FinMientras;
Para tabla <- tablaInicial Hasta tablaFinal Con Paso 1 Hacer
    Escribir "TABLA DEL ", tabla;
    Para multiplicador <- 1 Hasta multiplicadorFinal Con Paso 1 Hacer
        resultado <- tabla * multiplicador;
        Escribir tabla, " x ", multiplicador, " = ", resultado;
    FinPara;
FinPara;
```

FinAlgoritmo

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Tablas del 3 al 5, hasta el multiplicador 10

Variable	Valor
tablaInicial	3
tablaFinal	5
multiplicadorFinal	10
tablas generadas	3, 4 y 5
cantidad de operaciones	$3 \times 10 = 30$
último resultado	$5 \times 10 = 50$

Operaciones:

Tabla 3: $3 \times 1 = 3$; ...; $3 \times 10 = 30$.

Tabla 4: $4 \times 1 = 4$; ...; $4 \times 10 = 40$.

Tabla 5: $5 \times 1 = 5$; ...; $5 \times 10 = 50$.

Cantidad de tablas = $5 - 3 + 1 = 3$; operaciones = $3 \times 10 = 30$.

Caso 2: Intervalo inválido y corrección

Variable	Valor
primer intervalo	6 a 4 (inválido)
intervalo corregido	2 a 4
multiplicadorFinal	5
tablas generadas	2, 3 y 4
cantidad de operaciones	$3 \times 5 = 15$
último resultado	$4 \times 5 = 20$

Operaciones:

$6 > 4$, por lo tanto se muestra el error y se vuelven a leer los límites.

El intervalo 2 a 4 es válido porque $2 \leq 4$.

Tabla 2: $2 \times 1 = 2$; ...; $2 \times 5 = 10$.

Tabla 3: $3 \times 1 = 3$; ...; $3 \times 5 = 15$.

Tabla 4: $4 \times 1 = 4$; ...; $4 \times 5 = 20$.

Caso 3: Una sola tabla y validación del multiplicador

Variable	Valor
tablaInicial	7
tablaFinal	7
primer multiplicador final	0 (inválido)
multiplicador corregido	12
tablas generadas	7
cantidad de operaciones	12

Variable	Valor
último resultado	$7 \times 12 = 84$

Operaciones:

$7 \leq 7$, por lo tanto el intervalo de una sola tabla es válido.

0 no es mayor que cero; se solicita nuevamente y se acepta 12.

$7 \times 1 = 7$; $7 \times 2 = 14$; ...; $7 \times 12 = 84$.

Resumen de los casos

Caso	Intervalo	Multiplicador final	Tablas	Operaciones	Último resultado
1	3 a 5	10	3	30	$5 \times 10 = 50$
2	6 a 4 $\rightarrow$ 2 a 4	5	3	15	$4 \times 5 = 20$
3	7 a 7	0 $\rightarrow$ 12	1	12	$7 \times 12 = 84$

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class TablaMultiplicarConfigurable {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int tablaInicial, tablaFinal, multiplicadorFinal;
        int tabla, multiplicador, resultado;

        // Se leen los límites de las tablas hasta que sean correctos.
        System.out.print("Ingrese la tabla inicial: ");
        tablaInicial = entrada.nextInt();
        System.out.print("Ingrese la tabla final: ");
        tablaFinal = entrada.nextInt();

        while (tablaInicial > tablaFinal) {
            System.out.println("Error: la tabla inicial no puede ser mayor que la tabla final.");
            System.out.print("Ingrese nuevamente la tabla inicial: ");
            tablaInicial = entrada.nextInt();
            System.out.print("Ingrese nuevamente la tabla final: ");
            tablaFinal = entrada.nextInt();
        }

        // El usuario también decide hasta qué multiplicador se genera.
        System.out.print("Ingrese el multiplicador final (mayor que 0): ");
        multiplicadorFinal = entrada.nextInt();
        while (multiplicadorFinal <= 0) {
            System.out.println("Error: el multiplicador final debe ser mayor que 0.");
            System.out.print("Ingrese nuevamente el multiplicador final: ");
            multiplicadorFinal = entrada.nextInt();
        }

        // El primer for cambia de tabla y el segundo calcula sus productos.
        for (tabla = tablaInicial; tabla <= tablaFinal; tabla++) {
            System.out.println("\nTABLA DEL " + tabla);
            for (multiplicador = 1; multiplicador <= multiplicadorFinal; multiplicador++) {
                resultado = tabla * multiplicador;
                System.out.println(tabla + " x " + multiplicador + " = " + resultado);
            }
        }

        entrada.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int tablaInicial, tablaFinal, multiplicadorFinal;
    int tabla, multiplicador, resultado;

    // Se leen los límites de las tablas hasta que sean correctos.
    cout << "Ingrese la tabla inicial: ";
    cin >> tablaInicial;
    cout << "Ingrese la tabla final: ";
    cin >> tablaFinal;

    while (tablaInicial > tablaFinal) {
        cout << "Error: la tabla inicial no puede ser mayor que la tabla final.\n";
        cout << "Ingrese nuevamente la tabla inicial: ";
        cin >> tablaInicial;
        cout << "Ingrese nuevamente la tabla final: ";
        cin >> tablaFinal;
    }

    // El usuario también decide hasta qué multiplicador se genera.
    cout << "Ingrese el multiplicador final (mayor que 0): ";
    cin >> multiplicadorFinal;
    while (multiplicadorFinal <= 0) {
        cout << "Error: el multiplicador final debe ser mayor que 0.\n";
        cout << "Ingrese nuevamente el multiplicador final: ";
        cin >> multiplicadorFinal;
    }

    // El primer for cambia de tabla y el segundo calcula sus productos.
    for (tabla = tablaInicial; tabla <= tablaFinal; tabla++) {
        cout << "\nTABLA DEL " << tabla << "\n";
        for (multiplicador = 1; multiplicador <= multiplicadorFinal; multiplicador++) {
            resultado = tabla * multiplicador;
            cout << tabla << " x " << multiplicador << " = " << resultado << "\n";
        }
    }

    return 0;
}
```

SERIE Y SUMA DE NÚMEROS PARES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicitar un número entero positivo N. El programa debe mostrar todos los números pares desde 2 hasta N y calcular la cantidad de pares, su suma y su promedio. Se deben utilizar un contador, un acumulador, una validación y una estructura for. Como casos límite se comprobarán $N = 1$, $N = 2$ y valores negativos.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- N: número entero positivo mayor que cero que establece el límite superior de la serie.

1.2 Proceso

- Validar con while que N sea mayor que cero.
- Inicializar el contador y el acumulador suma en cero.
- Recorrer con un ciclo for los números pares desde 2 hasta N, avanzando de dos en dos.
- Mostrar cada número par, incrementar el contador y acumularlo en la suma.
- Si el contador es mayor que cero, calcular promedio = suma / contador.
- Si no existen números pares, evitar la división entre cero e indicar que el promedio no puede calcularse.

1.3 Salida de datos

- Serie de números pares desde 2 hasta N.
- Cantidad de números pares encontrados.
- Suma de los números pares.
- Promedio de los números pares, cuando exista al menos uno.
- Mensajes de validación y mensaje especial cuando $N = 1$.

2. ALGORITMO

1. **Inicio.**
2. Declarar N, numero, contador y suma como enteros, y promedio como real.
3. Solicitar el valor de N.
4. Mientras N sea menor o igual que cero, mostrar un mensaje de error y solicitarlo nuevamente.
5. Inicializar contador y suma en cero.
6. Recorrer con un ciclo for desde numero = 2 hasta N, aumentando numero de dos en dos.
7. Mostrar el número par actual.
8. Incrementar contador en uno y sumar el número actual al acumulador.
9. Al finalizar el ciclo, verificar si contador es mayor que cero.

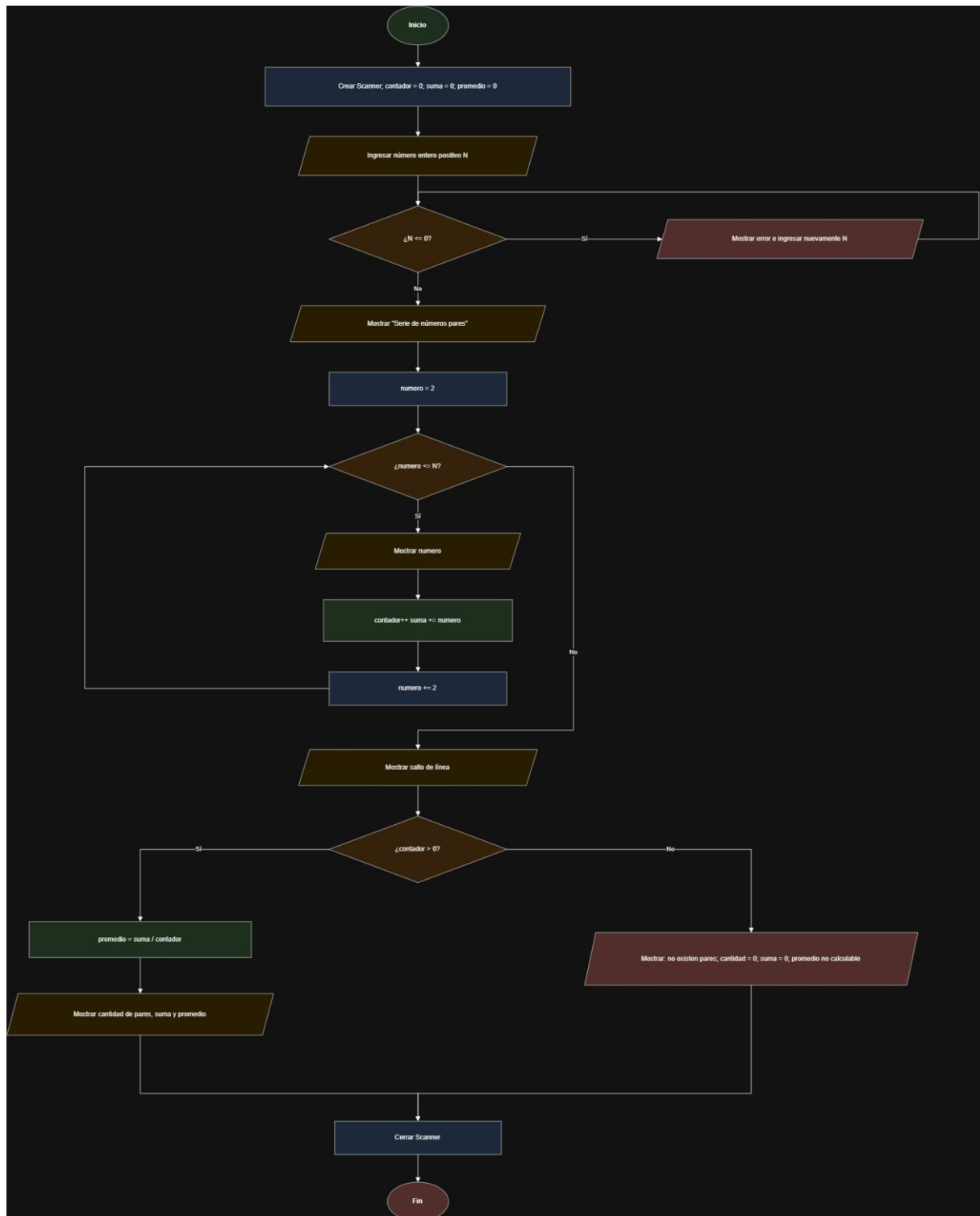
10. Si existen pares, calcular el promedio dividiendo suma para contador y mostrar los resultados.
11. Si no existen pares, mostrar cantidad y suma iguales a cero e indicar que el promedio no puede calcularse.
12. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo SerieSumaPares

```
Definir N, numero, contador, suma Como Entero;
Definir promedio Como Real;
Escribir "Ingrese un número entero positivo N:";
Leer N;
Mientras N <= 0 Hacer
    Escribir "Error: N debe ser mayor que cero";
    Escribir "Ingrese nuevamente N:";
    Leer N;
FinMientras;
contador <- 0;
suma <- 0;
Escribir "Serie de números pares:";
Para numero <- 2 Hasta N Con Paso 2 Hacer
    Escribir Sin Saltar numero, " ";
    contador <- contador + 1;
    suma <- suma + numero;
FinPara;
Escribir "";
Si contador > 0 Entonces
    promedio <- suma / contador;
    Escribir "Cantidad de pares: ", contador;
    Escribir "Suma: ", suma;
    Escribir "Promedio: ", promedio;
SiNo
    Escribir "No existen números pares entre 2 y ", N;
    Escribir "Cantidad de pares: 0";
    Escribir "Suma: 0";
    Escribir "Promedio: no se puede calcular";
FinSi;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Valor normal $N = 12$

Variable	Valor
N	12
serie	2, 4, 6, 8, 10, 12
contador	6
suma	42
promedio	7,00

Operaciones:

Iteración 1: numero = 2; contador = 1; suma = $0 + 2 = 2$.

Iteración 2: numero = 4; contador = 2; suma = $2 + 4 = 6$.

Iteración 3: numero = 6; contador = 3; suma = $6 + 6 = 12$.

Iteración 4: numero = 8; contador = 4; suma = $12 + 8 = 20$.

Iteración 5: numero = 10; contador = 5; suma = $20 + 10 = 30$.

Iteración 6: numero = 12; contador = 6; suma = $30 + 12 = 42$.

promedio = $42 / 6 = 7,00$.

Caso 2: Caso límite $N = 1$

Variable	Valor
N	1
serie	No existen números pares
contador	0
suma	0
promedio	No se calcula

Operaciones:

El ciclo inicia con numero = 2.

La condición $2 \leq 1$ es falsa, por lo tanto el ciclo no se ejecuta.

Como contador = 0, se evita dividir para cero y se informa que el promedio no puede calcularse.

Caso 3: Valor negativo y corrección a $N = 2$

Variable	Valor
primer N	-5 (inválido)
N corregido	2
serie	2
contador	1
suma	2
promedio	2,00

Operaciones:

$N = -5$ no cumple $N > 0$; se muestra el error y se solicita nuevamente.

Con $N = 2$: numero = 2; contador = 1; suma = $0 + 2 = 2$.

promedio = $2 / 1 = 2,00$.

Resumen de los casos

Caso	N ingresado	N válido	Serie	Suma	Promedio
1	12	12	2, 4, 6, 8, 10, 12	42	7,00
2	1	1	Sin pares	0	No se calcula
3	-5	2	2	2	2,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class SerieSumaPares {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n, numero, contador = 0, suma = 0;
        double promedio = 0;

        System.out.print("Ingrese un número entero positivo N: ");
        n = entrada.nextInt();

        // Se valida que N sea positivo.
        while (n <= 0) {
            System.out.println("Error: N debe ser mayor que cero.");
            System.out.print("Ingrese nuevamente N: ");
            n = entrada.nextInt();
        }

        System.out.println("Serie de números pares:");
        // El ciclo empieza en 2 y avanza de dos en dos.
        for (numero = 2; numero <= n; numero += 2) {
            System.out.print(numero + " ");
            contador++;
            suma += numero;
        }
        System.out.println();

        if (contador > 0) {
            promedio = (double) suma / contador;
            System.out.println("Cantidad de pares: " + contador);
            System.out.println("Suma: " + suma);
            System.out.printf("Promedio: %.2f%n", promedio);
        } else {
            System.out.println("No existen números pares entre 2 y " + n + ".");
            System.out.println("Cantidad de pares: 0");
            System.out.println("Suma: 0");
            System.out.println("Promedio: no se puede calcular.");
        }

        entrada.close();
    }
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n, numero, contador = 0, suma = 0;
    double promedio = 0;

    cout << "Ingrese un número entero positivo N: ";
    cin >> n;

    // Se valida que N sea positivo.
    while (n <= 0) {
        cout << "Error: N debe ser mayor que cero.\n";
        cout << "Ingrese nuevamente N: ";
        cin >> n;
    }

    cout << "Serie de números pares:\n";
    // El ciclo empieza en 2 y avanza de dos en dos.
    for (numero = 2; numero <= n; numero += 2) {
        cout << numero << " ";
        contador++;
        suma += numero;
    }
    cout << "\n";

    if (contador > 0) {
        promedio = static_cast<double>(suma) / contador;
        cout << "Cantidad de pares: " << contador << "\n";
        cout << "Suma: " << suma << "\n";
        cout << fixed << setprecision(2) << "Promedio: " << promedio << "\n";
    } else {
        cout << "No existen números pares entre 2 y " << n << ".\n";
        cout << "Cantidad de pares: 0\n";
        cout << "Suma: 0\n";
        cout << "Promedio: no se puede calcular.\n";
    }

    return 0;
}
```

CAJERO AUTOMÁTICO BÁSICO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construir un cajero automático con las opciones: consultar saldo, depositar, retirar, mostrar movimientos y salir. El usuario comenzará con un saldo de \$100 y el programa permanecerá activo hasta seleccionar Salir. No se permitirán depósitos negativos, retiros negativos, retiros mayores que el saldo disponible ni opciones inexistentes. Al finalizar se mostrarán los depósitos y retiros realizados, el total depositado, el total retirado y el saldo final. Se utilizarán do-while, switch, contadores y acumuladores.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Opción del menú: entero del 1 al 5.
- Monto de depósito: dato real mayor que cero.
- Monto de retiro: dato real mayor que cero y menor o igual que el saldo disponible.

1.2 Proceso

- Inicializar el saldo en \$100, los dos contadores en cero y los dos acumuladores en cero.
- Mostrar el menú al menos una vez mediante do-while y evaluar la opción con switch.
- Consultar y mostrar el saldo actual.
- Validar que un depósito sea positivo; si es válido, sumarlo al saldo, incrementar su contador y acumular su total.
- Validar que un retiro sea positivo y no exceda el saldo; si es válido, restarlo, incrementar su contador y acumular su total.
- Mostrar los movimientos acumulados cuando se seleccione la opción 4.
- Rechazar opciones diferentes de 1 a 5.
- Repetir el menú hasta seleccionar 5 y luego mostrar el resumen final.

1.3 Salida de datos

- Saldo actual.
- Mensajes de operación aceptada o rechazada.
- Cantidad de depósitos y retiros realizados.
- Total depositado y total retirado.
- Saldo final.
- Mensaje de opción inexistente.

2. ALGORITMO

1. **Inicio.**
2. Declarar las variables de opción, saldo, monto, contadores y acumuladores.
3. Asignar \$100 al saldo e inicializar contadores y acumuladores en cero.

4. Mostrar el menú y leer la opción.
5. Si la opción es 1, mostrar el saldo.
6. Si es 2, leer el depósito y aceptarlo únicamente si es positivo; actualizar saldo, contador y total depositado.
7. Si es 3, leer el retiro y comprobar que sea positivo y no exceda el saldo; actualizar saldo, contador y total retirado.
8. Si es 4, mostrar los movimientos y valores acumulados.
9. Si es 5, mostrar el mensaje de finalización.
10. Si la opción no existe, mostrar un error.
11. Repetir el menú mientras la opción sea diferente de 5.
12. Mostrar el resumen final completo.
13. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo CajeroAutomaticoConMovimientos

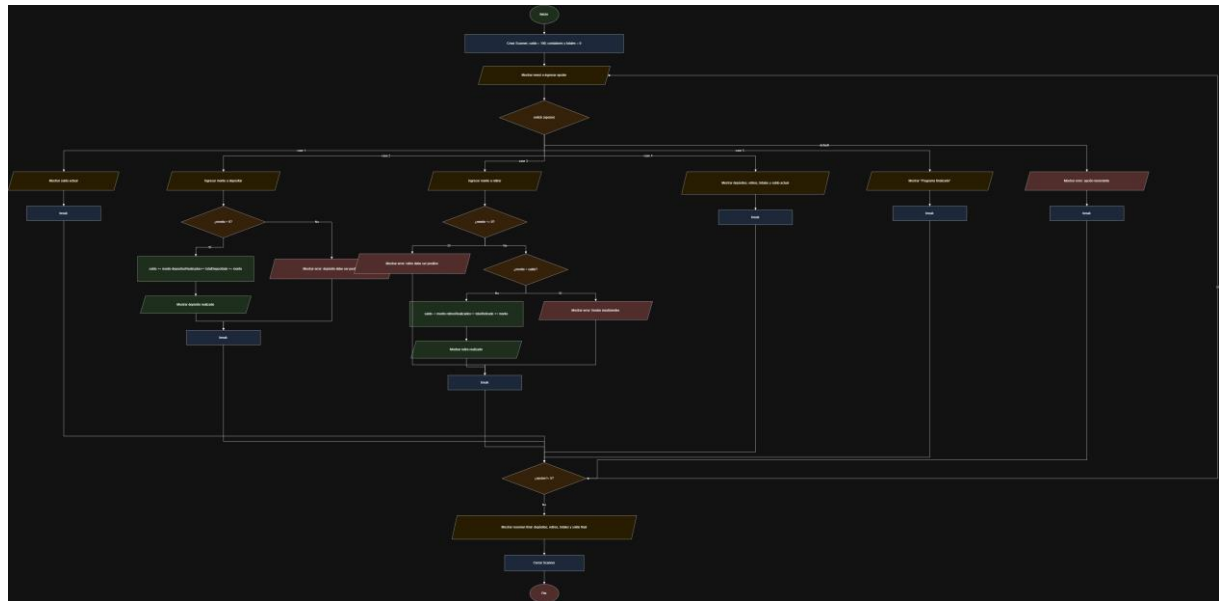
```

Definir opcion, depositosRealizados, retirosRealizados Como Entero;
Definir saldo, monto, totalDepositado, totalRetirado Como Real;
saldo <- 100;
depositosRealizados <- 0;
retirosRealizados <- 0;
totalDepositado <- 0;
totalRetirado <- 0;
Repetir
    Escribir "1. Consultar saldo";
    Escribir "2. Depositar";
    Escribir "3. Retirar";
    Escribir "4. Mostrar movimientos";
    Escribir "5. Salir";
    Leer opcion;
    Segun opcion Hacer
        1:
            Escribir "Saldo actual: $", saldo;
        2:
            Escribir "Monto a depositar:";
            Leer monto;
            Si monto > 0 Entonces
                saldo <- saldo + monto;
                depositosRealizados <- depositosRealizados + 1;
                totalDepositado <- totalDepositado + monto;
            SiNo
                Escribir "Error: el depósito debe ser positivo";
            FinSi;
        3:
            Escribir "Monto a retirar:";
            Leer monto;
            Si monto <= 0 Entonces
                Escribir "Error: el retiro debe ser positivo";
            SiNo
                Si monto > saldo Entonces
                    Escribir "Error: fondos insuficientes";
                SiNo
                    saldo <- saldo - monto;
                    retirosRealizados <- retirosRealizados + 1;
                    totalRetirado <- totalRetirado + monto;
                FinSi;
            FinSi;
    FinRepetir

```

```
    FinSi;
4:
    Escribir "Depósitos realizados: ", depositosRealizados;
    Escribir "Retiros realizados: ", retirosRealizados;
    Escribir "Total depositado: $", totalDepositado;
    Escribir "Total retirado: $", totalRetirado;
    Escribir "Saldo actual: $", saldo;
5:
    Escribir "Programa finalizado";
De Otro Modo:
    Escribir "Error: opción inexistente";
FinSegun;
Hasta Que opcion = 5;
Escribir "Depósitos realizados: ", depositosRealizados;
Escribir "Retiros realizados: ", retirosRealizados;
Escribir "Total depositado: $", totalDepositado;
Escribir "Total retirado: $", totalRetirado;
Escribir "Saldo final: $", saldo;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Depósito y retiro válidos

Variable	Valor
saldo inicial	\$100,00
depósito	\$50,00
retiro	\$30,00
depósitos realizados	1
retiros realizados	1
total depositado	\$50,00
total retirado	\$30,00
saldo final	\$120,00

Operaciones:

Después del depósito: saldo = $100 + 50 = \$150,00$.

Después del retiro: saldo = $150 - 30 = \$120,00$.

Los dos movimientos son válidos y actualizan sus contadores y acumuladores.

Caso 2: Montos negativos rechazados

Variable	Valor
saldo inicial	\$100,00
depósito intentado	-\$20,00
retiro intentado	-\$10,00
depósitos realizados	0
retiros realizados	0
total depositado	\$0,00
total retirado	\$0,00
saldo final	\$100,00

Operaciones:

$-20 \leq 0$: el depósito se rechaza y no modifica ningún valor.

$-10 \leq 0$: el retiro se rechaza y no modifica ningún valor.

Saldo final = \$100,00.

Caso 3: Fondos insuficientes y opción inexistente

Variable	Valor
saldo inicial	\$100,00
opción intentada	9 (inexistente)
retiro intentado	\$150,00
depósito válido	\$25,00
depósitos realizados	1

Variable	Valor
retiros realizados	0
total depositado	\$25,00
total retirado	\$0,00
saldo final	\$125,00

Operaciones:

La opción 9 se rechaza y el menú se muestra nuevamente.

\$150,00 > \$100,00: el retiro se rechaza por fondos insuficientes.

Depósito válido: saldo = 100 + 25 = \$125,00.

Resumen de los casos

Caso	Depósitos	Retiros	Total depositado	Total retirado	Saldo final
1	1	1	\$50,00	\$30,00	\$120,00
2	0	0	\$0,00	\$0,00	\$100,00
3	1	0	\$25,00	\$0,00	\$125,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class CajeroMovimientos {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int opcion, depositosRealizados = 0, retirosRealizados = 0;
        double saldo = 100, monto;
        double totalDepositado = 0, totalRetirado = 0;

        do {
            System.out.println("\n===== CAJERO AUTOMÁTICO =====");
            System.out.println("1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("4. Mostrar movimientos");
            System.out.println("5. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = entrada.nextInt();

            switch (opcion) {
                case 1:
                    System.out.printf("Saldo actual: $%.2f\n", saldo);
                    break;
                case 2:
                    System.out.print("Monto a depositar: $");
                    monto = entrada.nextDouble();
                    if (monto > 0) {
                        saldo += monto;
                        depositosRealizados++;
                        totalDepositado += monto;
                        System.out.println("Depósito realizado correctamente.");
                    } else {
                        System.out.println("Error: el depósito debe ser positivo.");
                    }
                    break;
                case 3:
                    System.out.print("Monto a retirar: $");
                    monto = entrada.nextDouble();
                    if (monto <= 0) {
                        System.out.println("Error: el retiro debe ser positivo.");
                    } else if (monto > saldo) {
                        System.out.println("Error: fondos insuficientes.");
                    } else {
                        saldo -= monto;
                        retirosRealizados++;
                        totalRetirado += monto;
                        System.out.println("Retiro realizado correctamente.");
                    }
                    break;
                case 4:
                    System.out.println("Depósitos realizados: " + depositosRealizados);
                    System.out.println("Retiros realizados: " + retirosRealizados);
                    System.out.printf("Total depositado: $%.2f\n", totalDepositado);
                    System.out.printf("Total retirado: $%.2f\n", totalRetirado);
                    System.out.printf("Saldo actual: $%.2f\n", saldo);
                    break;
                case 5:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Error: opción inexistente.");
            }
        } while (opcion != 5);
    }
}
```

```
System.out.println("\n===== RESUMEN FINAL =====");
System.out.println("Depósitos realizados: " + depositosRealizados);
System.out.println("Retiros realizados: " + retirosRealizados);
System.out.printf("Total depositado: $%.2f%n", totalDepositado);
System.out.printf("Total retirado: $%.2f%n", totalRetirado);
System.out.printf("Saldo final: $%.2f%n", saldo);
entrada.close();
}
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int opcion, depositosRealizados = 0, retirosRealizados = 0;
    double saldo = 100, monto;
    double totalDepositado = 0, totalRetirado = 0;

    do {
        cout << "\n===== CAJERO AUTOMATICO =====\n";
        cout << "1. Consultar saldo\n2. Depositar\n3. Retirar\n";
        cout << "4. Mostrar movimientos\n5. Salir\n";
        cout << "Seleccione una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1:
                cout << fixed << setprecision(2) << "Saldo actual: $" << saldo << "\n";
                break;
            case 2:
                cout << "Monto a depositar: $";
                cin >> monto;
                if (monto > 0) {
                    saldo += monto;
                    depositosRealizados++;
                    totalDepositado += monto;
                    cout << "Deposito realizado correctamente.\n";
                } else {
                    cout << "Error: el deposito debe ser positivo.\n";
                }
                break;
            case 3:
                cout << "Monto a retirar: $";
                cin >> monto;
                if (monto <= 0) {
                    cout << "Error: el retiro debe ser positivo.\n";
                } else if (monto > saldo) {
                    cout << "Error: fondos insuficientes.\n";
                } else {
                    saldo -= monto;
                    retirosRealizados++;
                    totalRetirado += monto;
                    cout << "Retiro realizado correctamente.\n";
                }
                break;
            case 4:
                cout << "Depositos realizados: " << depositosRealizados << "\n";
                cout << "Retiros realizados: " << retirosRealizados << "\n";
                cout << fixed << setprecision(2);
                cout << "Total depositado: $" << totalDepositado << "\n";
                cout << "Total retirado: $" << totalRetirado << "\n";
                cout << "Saldo actual: $" << saldo << "\n";
                break;
            case 5:
                cout << "Programa finalizado.\n";
                break;
            default:
                cout << "Error: opcion inexistente.\n";
        }
    } while (opcion != 5);

    cout << "\n===== RESUMEN FINAL =====\n";
    cout << "Depositos realizados: " << depositosRealizados << "\n";
    cout << "Retiros realizados: " << retirosRealizados << "\n";
```

```
cout << fixed << setprecision(2);  
cout << "Total depositado: $" << totalDepositado << "\n";  
cout << "Total retirado: $" << totalRetirado << "\n";  
cout << "Saldo final: $" << saldo << "\n";  
return 0;  
}
```

ESTACIONAMIENTO UNIVERSITARIO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Desarrollar un programa para registrar varios vehículos que utilizan un estacionamiento universitario. Por cada vehículo se solicitará el tipo: motocicleta, automóvil o camioneta, y las horas estacionado. Las tarifas por hora son \$0,50 para motocicleta, \$1,00 para automóvil y \$1,50 para camioneta. El registro continuará hasta que el usuario decida terminar. Al finalizar se mostrará la cantidad por tipo, el total de vehículos, la recaudación total y el promedio pagado. Las horas deben ser mayores que cero y se utilizarán do-while, switch, contadores y acumuladores.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Tipo de vehículo: entero 1 para motocicleta, 2 para automóvil o 3 para camioneta.
- Horas estacionado: entero mayor que cero.
- Respuesta para continuar: 1 para registrar otro vehículo o 0 para terminar.

1.2 Proceso

- Inicializar en cero los contadores de cada tipo, el total de vehículos y la recaudación.
- Validar que el tipo de vehículo esté entre 1 y 3.
- Validar mediante do-while que las horas sean mayores que cero.
- Asignar con switch la tarifa correspondiente e incrementar el contador del tipo.
- Calcular $\text{pago} = \text{tarifa por hora} \times \text{horas}$.
- Incrementar el total de vehículos y acumular el pago en la recaudación.
- Preguntar si se registrará otro vehículo y validar la respuesta.
- Al terminar, calcular $\text{promedio pagado} = \text{total recaudado} / \text{total de vehículos}$.
- Evitar la división entre cero; aunque el ciclo registra al menos un vehículo, se conserva la validación de seguridad.

1.3 Salida de datos

- Valor que debe pagar cada vehículo.
- Mensajes de validación para tipo, horas o respuesta incorrectos.
- Cantidad de motocicletas, automóviles y camionetas.
- Total de vehículos registrados.
- Total recaudado y promedio pagado.

2. ALGORITMO

1. Inicio.

2. Declarar las variables para tipo, horas, continuar, tarifas, pagos, contadores y acumuladores.

3. Inicializar los contadores y la recaudación en cero.
4. Solicitar y validar el tipo de vehículo entre 1 y 3.
5. Solicitar y validar que las horas sean mayores que cero.
6. Evaluar el tipo mediante switch para asignar la tarifa e incrementar su contador.
7. Calcular el pago multiplicando la tarifa por las horas.
8. Incrementar el total de vehículos y acumular el pago.
9. Mostrar el valor a pagar.
10. Preguntar si desea registrar otro vehículo y validar 1 o 0.
11. Repetir el registro mientras la respuesta sea 1.
12. Calcular el promedio pagado dividiendo la recaudación para el total de vehículos.
13. Mostrar el reporte final.
14. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo EstacionamientoUniversitario

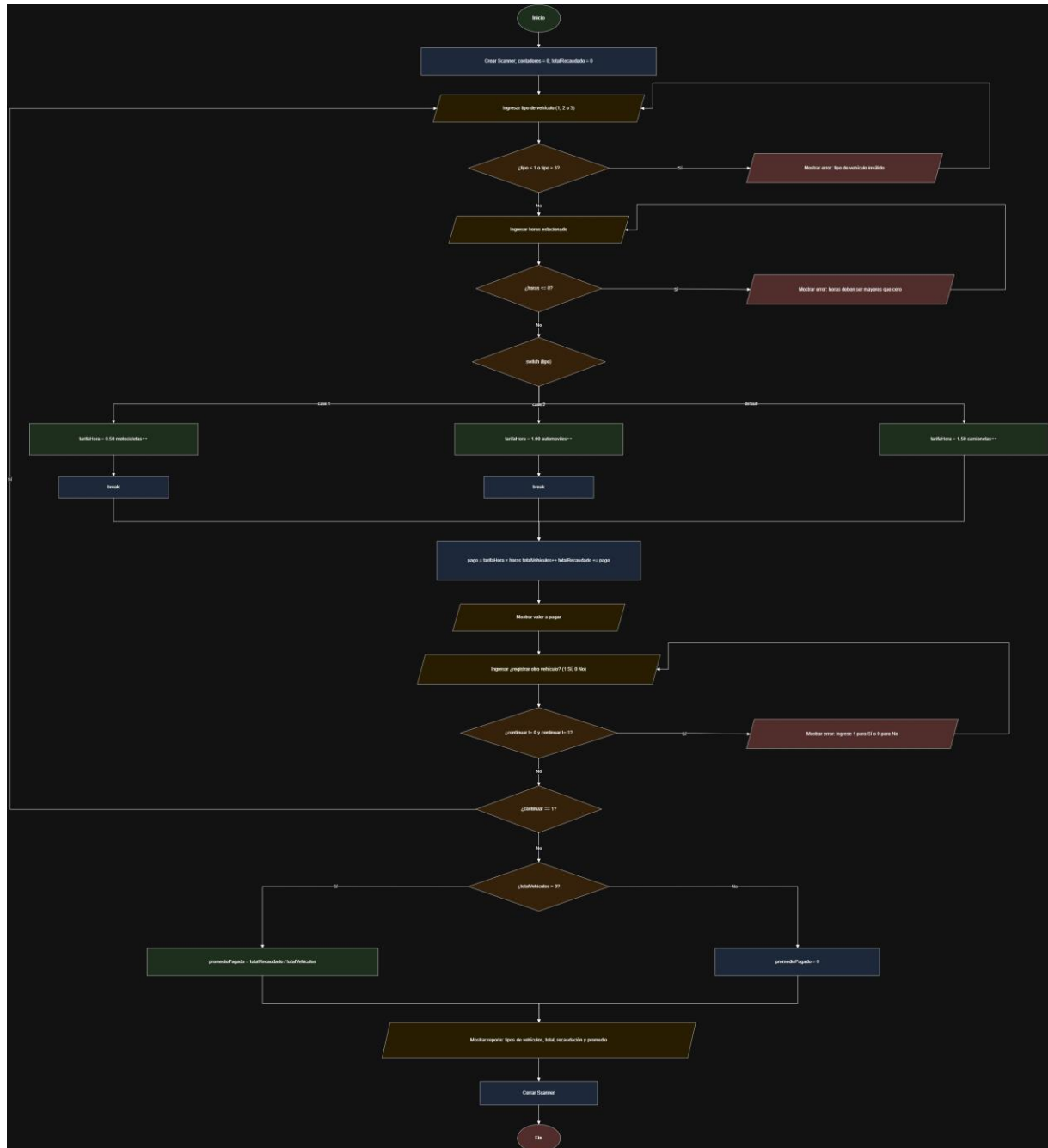
```

Definir tipo, horas, continuar Como Entero;
Definir motocicletas, automoviles, camionetas, totalVehiculos Como Entero;
Definir tarifaHora, pago, totalRecaudado, promedioPagado Como Real;
motocicletas <- 0;
automoviles <- 0;
camionetas <- 0;
totalVehiculos <- 0;
totalRecaudado <- 0;
Repetir
    Repetir
        Escribir "Tipo: 1 Motocicleta, 2 Automóvil, 3 Camioneta";
        Leer tipo;
        Si tipo < 1 O tipo > 3 Entonces
            Escribir "Error: tipo de vehículo inválido";
        FinSi;
    Hasta Que tipo >= 1 Y tipo <= 3;
    Repetir
        Escribir "Horas estacionado:";
        Leer horas;
        Si horas <= 0 Entonces
            Escribir "Error: las horas deben ser mayores que cero";
        FinSi;
    Hasta Que horas > 0;
    Segun tipo Hacer
        1:
            tarifaHora <- 0.50;
            motocicletas <- motocicletas + 1;
        2:
            tarifaHora <- 1.00;
            automoviles <- automoviles + 1;
        3:
            tarifaHora <- 1.50;
            camionetas <- camionetas + 1;
    FinSegun;
    pago <- tarifaHora * horas;
    totalVehiculos <- totalVehiculos + 1;
    totalRecaudado <- totalRecaudado + pago;
    Escribir "Valor a pagar: $", pago;

```

```
Repetir
  Escribir "¿Registrar otro vehículo? 1 Sí, 0 No";
  Leer continuar;
  Hasta Que continuar = 0 O continuar = 1;
Hasta Que continuar = 0;
Si totalVehiculos > 0 Entonces
  promedioPagado <- totalRecaudado / totalVehiculos;
SiNo
  promedioPagado <- 0;
FinSi;
Escribir "Motocicletas: ", motocicletas;
Escribir "Automóviles: ", automoviles;
Escribir "Camionetas: ", camionetas;
Escribir "Total vehículos: ", totalVehiculos;
Escribir "Total recaudado: $", totalRecaudado;
Escribir "Promedio pagado: $", promedioPagado;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Una motocicleta durante 4 horas

Variable	Valor
tipo	1 - Motocicleta
horas	4
tarifa	\$0,50
pago	\$2,00
motocicletas / total	1 / 1
recaudación	\$2,00
promedio	\$2,00

Operaciones:

$$\text{pago} = 0,50 \times 4 = \$2,00.$$

$$\text{promedio} = 2,00 / 1 = \$2,00.$$

Caso 2: Automóvil y camioneta

Variable	Valor
vehículo 1	Automóvil, 3 horas
pago 1	$\$1,00 \times 3 = \$3,00$
vehículo 2	Camioneta, 2 horas
pago 2	$\$1,50 \times 2 = \$3,00$
automóviles / camionetas	1 / 1
total vehículos	2
recaudación	\$6,00
promedio	\$3,00

Operaciones:

$$\text{total recaudado} = 3,00 + 3,00 = \$6,00.$$

$$\text{promedio} = 6,00 / 2 = \$3,00.$$

Caso 3: Validación de horas y los tres tipos

Variable	Valor
motocicleta	1 hora = \$0,50
automóvil	2 horas = \$2,00
camioneta	0 horas (inválido) $\rightarrow$ 3 horas = \$4,50
contadores	1 motocicleta, 1 automóvil, 1 camioneta
total vehículos	3
recaudación	\$7,00
promedio	\$2,33

Operaciones:

0 no es mayor que cero, por lo que las horas de la camioneta se solicitan nuevamente.

total recaudado = $0,50 + 2,00 + 4,50 = \$7,00$.

promedio = $7,00 / 3 = 2,333... \approx \$2,33$.

Resumen de los casos

Caso	Motos	Autos	Camionetas	Total	Recaudación	Promedio
1	1	0	0	1	\$2,00	\$2,00
2	0	1	1	2	\$6,00	\$3,00
3	1	1	1	3	\$7,00	\$2,33

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class EstacionamientoRegistro {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int tipo, horas, continuar;
        int motocicletas = 0, automoviles = 0, camionetas = 0;
        int totalVehiculos = 0;
        double tarifaHora = 0, pago, totalRecaudado = 0, promedioPagado;

        do {
            do {
                System.out.print("Tipo (1 Motocicleta, 2 Automóvil, 3 Camioneta): ");
                tipo = entrada.nextInt();
                if (tipo < 1 || tipo > 3) {
                    System.out.println("Error: tipo de vehículo inválido.");
                }
            } while (tipo < 1 || tipo > 3);

            do {
                System.out.print("Horas estacionado: ");
                horas = entrada.nextInt();
                if (horas <= 0) {
                    System.out.println("Error: las horas deben ser mayores que cero.");
                }
            } while (horas <= 0);

            switch (tipo) {
                case 1:
                    tarifaHora = 0.50;
                    motocicletas++;
                    break;
                case 2:
                    tarifaHora = 1.00;
                    automoviles++;
                    break;
                default:
                    tarifaHora = 1.50;
                    camionetas++;
            }

            pago = tarifaHora * horas;
            totalVehiculos++;
            totalRecaudado += pago;
            System.out.printf("Valor a pagar: $%.2f\n", pago);

            do {
                System.out.print("¿Registrar otro vehículo? (1 Sí, 0 No): ");
                continuar = entrada.nextInt();
                if (continuar != 0 && continuar != 1) {
                    System.out.println("Error: ingrese 1 para Sí o 0 para No.");
                }
            } while (continuar != 0 && continuar != 1);
        } while (continuar == 1);

        if (totalVehiculos > 0) {
            promedioPagado = totalRecaudado / totalVehiculos;
        } else {
            promedioPagado = 0;
        }

        System.out.println("\n===== REPORTE =====");
        System.out.println("Motocicletas: " + motocicletas);
    }
}
```

```
System.out.println("Automóviles: " + automoviles);
System.out.println("Camionetas: " + camionetas);
System.out.println("Total vehículos: " + totalVehiculos);
System.out.printf("Total recaudado: $%.2f%n", totalRecaudado);
System.out.printf("Promedio pagado: $%.2f%n", promedioPagado);
System.out.println("=====");
entrada.close();
}
}
```

C++

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int tipo, horas, continuar;
    int motocicletas = 0, automoviles = 0, camionetas = 0;
    int totalVehiculos = 0;
    double tarifaHora = 0, pago, totalRecaudado = 0, promedioPagado;

    do {
        do {
            cout << "Tipo (1 Motocicleta, 2 Automovil, 3 Camioneta): ";
            cin >> tipo;
            if (tipo < 1 || tipo > 3) cout << "Error: tipo de vehiculo invalido.\n";
        } while (tipo < 1 || tipo > 3);

        do {
            cout << "Horas estacionado: ";
            cin >> horas;
            if (horas <= 0) cout << "Error: las horas deben ser mayores que cero.\n";
        } while (horas <= 0);

        switch (tipo) {
            case 1: tarifaHora = 0.50; motocicletas++; break;
            case 2: tarifaHora = 1.00; automoviles++; break;
            default: tarifaHora = 1.50; camionetas++;
        }

        pago = tarifaHora * horas;
        totalVehiculos++;
        totalRecaudado += pago;
        cout << fixed << setprecision(2) << "Valor a pagar: $" << pago << "\n";

        do {
            cout << "Registrar otro vehiculo? (1 Si, 0 No): ";
            cin >> continuar;
            if (continuar != 0 && continuar != 1) cout << "Error: ingrese 1 para Si o 0 para No.\n";
        } while (continuar != 0 && continuar != 1);
    } while (continuar == 1);

    if (totalVehiculos > 0) promedioPagado = totalRecaudado / totalVehiculos;
    else promedioPagado = 0;

    cout << "\n===== REPORTE =====\n";
    cout << "Motocicletas: " << motocicletas << "\n";
    cout << "Automoviles: " << automoviles << "\n";
    cout << "Camionetas: " << camionetas << "\n";
    cout << "Total vehiculos: " << totalVehiculos << "\n";
    cout << fixed << setprecision(2);
    cout << "Total recaudado: $" << totalRecaudado << "\n";
    cout << "Promedio pagado: $" << promedioPagado << "\n";
    cout << "===== \n";
    return 0;
}
```

TRIÁNGULO Y PATRONES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Solicitar un número entero entre 2 y 10. Mostrar primero un triángulo ascendente de asteriscos, luego un triángulo descendente de asteriscos y finalmente un patrón numérico ascendente. Cada figura tendrá N filas. Se debe explicar mediante una tabla de trazas cómo interactúan el ciclo exterior y el ciclo interior. La estructura obligatoria son ciclos for anidados.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- N: número entero comprendido entre 2 y 10, incluidos ambos límites.

1.2 Proceso

- Validar que N se encuentre entre 2 y 10.
- Para el primer patrón, usar un for exterior con i desde 1 hasta N y un for interior con j desde 1 hasta i; imprimir un asterisco por cada repetición interior.
- Para el segundo patrón, usar el for exterior con i desde N hasta 1 y el interior con j desde 1 hasta i; imprimir asteriscos.
- Para el tercer patrón, recorrer i desde 1 hasta N y j desde 1 hasta i; imprimir el valor de j.
- Después de cada ciclo interior, realizar un salto de línea para iniciar la siguiente fila.

1.3 Salida de datos

- Mensaje de validación cuando N esté fuera del intervalo.
- Triángulo ascendente de asteriscos.
- Triángulo descendente de asteriscos.
- Patrón numérico ascendente.

2. ALGORITMO

1. **Inicio.**
2. Declarar N, i y j como enteros.
3. Solicitar N.
4. Mientras N sea menor que 2 o mayor que 10, mostrar un error y solicitarlo nuevamente.
5. Recorrer i desde 1 hasta N.
6. Para cada i, recorrer j desde 1 hasta i e imprimir un asterisco; luego realizar un salto de línea.
7. Recorrer i desde N hasta 1.
8. Para cada i, recorrer j desde 1 hasta i e imprimir un asterisco; luego realizar un salto de línea.
9. Recorrer i desde 1 hasta N.
10. Para cada i, recorrer j desde 1 hasta i e imprimir j; luego realizar un salto de línea.

11. **Fin.**

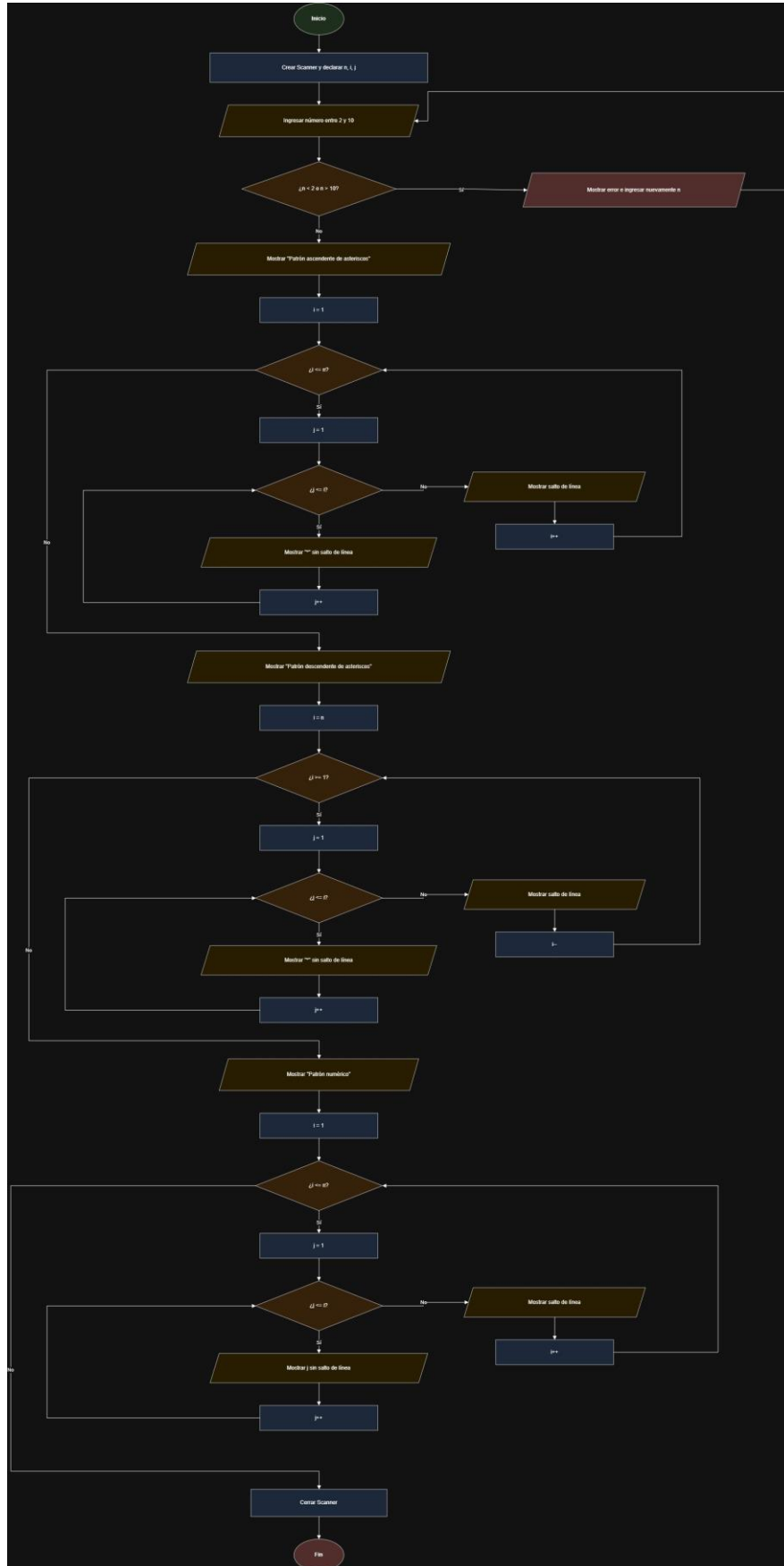
3. PSEUDOCÓDIGO

Algoritmo TrianguloYPatrones

```
Definir N, i, j Como Entero;  
Escribir "Ingrese un número entre 2 y 10:";  
Leer N;  
Mientras N < 2 O N > 10 Hacer  
    Escribir "Error: el número debe estar entre 2 y 10";  
    Escribir "Ingrese nuevamente:";  
    Leer N;  
FinMientras;  
Escribir "Patrón ascendente de asteriscos:";  
Para i <- 1 Hasta N Con Paso 1 Hacer  
    Para j <- 1 Hasta i Con Paso 1 Hacer  
        Escribir Sin Saltar "*";  
    FinPara;  
    Escribir "";  
FinPara;  
Escribir "Patrón descendente de asteriscos:";  
Para i <- N Hasta 1 Con Paso -1 Hacer  
    Para j <- 1 Hasta i Con Paso 1 Hacer  
        Escribir Sin Saltar "*";  
    FinPara;  
    Escribir "";  
FinPara;  
Escribir "Patrón numérico:";  
Para i <- 1 Hasta N Con Paso 1 Hacer  
    Para j <- 1 Hasta i Con Paso 1 Hacer  
        Escribir Sin Saltar j;  
    FinPara;  
    Escribir "";  
FinPara;
```

FinAlgoritmo

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1: Valor solicitado N = 5

Variable	Valor
N	5
patrón ascendente	* / ** / *** / **** / *****
patrón descendente	***** / **** / *** / ** / *
patrón numérico	1 / 12 / 123 / 1234 / 12345
filas por patrón	5
repeticiones internas por patrón	$1 + 2 + 3 + 4 + 5 = 15$

Operaciones:

El ciclo exterior crea cinco filas en cada patrón.

En cada fila, el límite del ciclo interior es i; por eso la cantidad de elementos coincide con el valor actual de i.

En el patrón descendente, i comienza en 5 y disminuye hasta 1.

Tabla de trazas de los ciclos anidados para N = 5

Patrón	Ciclo exterior i	Valores del ciclo interior j	Repeticiones internas	Salida de la fila
Ascendente	1	1	1	*
Ascendente	2	1, 2	2	**
Ascendente	3	1, 2, 3	3	***
Ascendente	4	1, 2, 3, 4	4	****
Ascendente	5	1, 2, 3, 4, 5	5	*****
Descendente	5	1, 2, 3, 4, 5	5	*****
Descendente	4	1, 2, 3, 4	4	****
Descendente	3	1, 2, 3	3	***
Descendente	2	1, 2	2	**
Descendente	1	1	1	*
Numérico	1	1	1	1
Numérico	2	1, 2	2	12
Numérico	3	1, 2, 3	3	123
Numérico	4	1, 2, 3, 4	4	1234
Numérico	5	1, 2, 3, 4, 5	5	12345

Caso 2: Límite inferior N = 2

Variable	Valor
N	2
ascendente	* / **
descendente	** / *
numérico	1 / 12
repeticiones internas por patrón	$1 + 2 = 3$

Operaciones:

Fila 1: i = 1, el ciclo interior se ejecuta una vez.

Fila 2: $i = 2$, el ciclo interior se ejecuta dos veces.

El valor 2 es aceptado porque pertenece al intervalo $[2, 10]$.

Caso 3: Valor inválido 11 y corrección a 10

Variable	Valor
primer N	11 (inválido)
N corregido	10
filas por patrón	10
última fila ascendente	*****
primera fila descendente	*****
última fila numérica	12345678910
repeticiones internas por patrón	55

Operaciones:

$11 > 10$: se muestra el error y se solicita nuevamente.

Con $N = 10$, las repeticiones internas son $1 + 2 + \dots + 10 = 10 \times 11 / 2 = 55$ por patrón.

Entre los tres patrones se realizan $55 \times 3 = 165$ impresiones internas.

Resumen de los casos

Caso	Entrada	N válido	Filas por patrón	Repeticiones internas por patrón
1	5	5	5	15
2	2	2	2	3
3	$11 \rightarrow 10$	10	10	55

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class TrianguloPatrones {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n, i, j;

        System.out.print("Ingrese un número entre 2 y 10: ");
        n = entrada.nextInt();
        while (n < 2 || n > 10) {
            System.out.println("Error: el número debe estar entre 2 y 10.");
            System.out.print("Ingrese nuevamente: ");
            n = entrada.nextInt();
        }

        System.out.println("\nPatrón ascendente de asteriscos:");
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nPatrón descendente de asteriscos:");
        for (i = n; i >= 1; i--) {
            for (j = 1; j <= i; j++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nPatrón numérico:");
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= i; j++) {
                System.out.print(j);
            }
            System.out.println();
        }

        entrada.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int n, i, j;

    cout << "Ingrese un numero entre 2 y 10: ";
    cin >> n;
    while (n < 2 || n > 10) {
        cout << "Error: el numero debe estar entre 2 y 10.\n";
        cout << "Ingrese nuevamente: ";
        cin >> n;
    }

    cout << "\nPatron ascendente de asteriscos:\n";
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    cout << "\nPatron descendente de asteriscos:\n";
    for (i = n; i >= 1; i--) {
        for (j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << "\n";
    }

    cout << "\nPatron numerico:\n";
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++) {
            cout << j;
        }
        cout << "\n";
    }

    return 0;
}
```

CONTROL DE VENTAS DE CAFETERÍA

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construya un sistema con el menú CAFETERÍA UNIVERSITARIA: 1. Registrar venta; 2. Mostrar estadísticas; 3. Mostrar tabla de productos; 4. Salir. Productos: 1. Café \$1.00; 2. Sándwich \$2.50; 3. Jugo \$1.50; 4. Empanada \$1.25. Al registrar una venta deberá solicitar producto y cantidad. El reporte deberá indicar número de ventas, cantidad total de productos, total recaudado, promedio por venta y producto con mayor cantidad vendida. Validar todas las entradas. Estructuras esperadas: do-while, switch, contadores, acumuladores y condicionales.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- Opción: entero entre 1 y 4. Producto: entero entre 1 y 4, con la correspondencia del enunciado.
- Cantidad: entero positivo; se admite hasta 2147483647 por el tipo int de la implementación. Ingresar solo dígitos, sin signos, letras, espacios ni decimales.

1.2 Proceso

- Iniciar ventas, unidades, cantidades por producto y recaudación en cero. Mostrar y repetir el menú con do-while.
- Leer cada entrada como texto y revisar sus dígitos antes de convertirla. Repetir si está vacía, contiene caracteres no numéricos, es cero o excede el rango.
- En la opción 1, validar producto y cantidad. Con switch asignar precio y sumar cantidad al contador del producto. Calcular importe = precio $\times$ cantidad; aumentar ventas en uno, unidades en cantidad y total en importe.
- En la opción 2, mostrar ventas, unidades y recaudación. Si ventas es cero, indicar que no hay promedio ni producto líder; si no, calcular total / ventas.
- Comparar los cuatro contadores para hallar el mayor. Mostrar todos los productos que igualen ese valor, incluidos los empates.
- En la opción 3, mostrar los cuatro productos y sus precios, sin modificar estadísticas. En la opción 4, finalizar.

1.3 Salida de datos

- Menú, solicitudes de datos y mensajes de entrada inválida.
- Importe de cada venta válida. Una venta es un registro de un producto y su cantidad, no una unidad individual.
- Estadísticas solicitadas, máximo de unidades vendidas y nombres de los líderes; mensaje especial cuando aún no hay ventas.
- Tabla de productos y mensaje de finalización. Importes en Java y C++ con dos decimales.

2. ALGORITMO

1. **Inicio.**
2. Declarar variables escalares y poner a cero los contadores y el total recaudado.
3. Mostrar el menú y leer la opción como texto.
4. Validar cada entrada: iniciar dato = 0 y revisar cada carácter; si es un dígito, actualizar dato = dato $\times$ 10 + dígito. Si no es dígito o excede el máximo, invalidar. Rechazar también texto vacío y dato menor que 1. Repetir la lectura hasta aceptar.
5. Para la opción 1, solicitar producto (1 a 4) y cantidad (1 a 2147483647), aplicando la misma validación.
6. Evaluar producto con switch; asignar su precio y aumentar su contador en la cantidad vendida.
7. Calcular importe = precio $\times$ cantidad; aumentar ventas en 1, unidades en cantidad y total en importe. Mostrar el importe.
8. Para la opción 2, mostrar ventas, unidades y total. Si ventas = 0, informar que promedio y líder no están disponibles.
9. Si hay ventas, calcular promedio = total / ventas. Inicializar mayor con café y compararlo sucesivamente con sándwich, jugo y empanada.
10. Mostrar el promedio, la cantidad máxima y cada producto cuyo contador sea igual a mayor.
11. Para la opción 3, mostrar la tabla de productos sin cambiar acumuladores. Para la opción 4, mostrar la despedida.
12. Repetir el menú mientras la opción sea diferente de 4.
13. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo ControlVentasCafeteria

```
Definir opcion, producto, cantidad, posicion, digito, d Como Entero;
Definir ventas, unidades, cafe, sandwich, jugo, empanada, mayor Como Entero;
Definir dato, precio, importe, total, promedio Como Real;
Definir texto, caracter Como Cadena;
Definir valido Como Logico;
ventas <- 0;
unidades <- 0;
cafe <- 0;
sandwich <- 0;
jugo <- 0;
empanada <- 0;
total <- 0;
Repetir
    Escribir "===== CAFETERIA UNIVERSITARIA =====";
    Escribir "1. Registrar venta";
    Escribir "2. Mostrar estadísticas";
    Escribir "3. Mostrar tabla de productos";
    Escribir "4. Salir";
Repetir
    Escribir "Opcion (1 a 4):";
    Leer texto;
    valido <- Longitud(texto) > 0;
    dato <- 0;
    posicion <- 0;
    Mientras posicion < Longitud(texto) Y valido Hacer
        caracter <- Subcadena(texto, posicion, posicion);
        digito <- -1;
        Para d <- 0 Hasta 9 Hacer
            Si caracter = ConvertirATexto(d) Entonces
                digito <- d;
            FinSi;
        FinPara;
        Si digito = -1 Entonces
            valido <- Falso;
        SiNo
            dato <- dato * 10 + digito;
            Si dato > 4 Entonces
                valido <- Falso;
            FinSi;
        FinSi;
        posicion <- posicion + 1;
    FinMientras;
    Si dato < 1 Entonces
        valido <- Falso;
    FinSi;
    Si NO valido Entonces
        Escribir "Entrada invalida. Intente otra vez.";
    FinSi;
Hasta Que valido;
opcion <- dato;
Segun opcion Hacer
    1:
```

Repetir

Escribir "Producto: 1 Cafe, 2 Sandwich, 3 Jugo, 4 Empanada";

Leer texto;

valido <- Longitud(texto) > 0;

dato <- 0;

posicion <- 0;

Mientras posicion < Longitud(texto) Y valido Hacer

caracter <- Subcadena(texto, posicion, posicion);

digito <- -1;

Para d <- 0 Hasta 9 Hacer

Si caracter = ConvertirATexto(d) Entonces

digito <- d;

FinSi;

FinPara;

Si digito = -1 Entonces

valido <- Falso;

SiNo

dato <- dato * 10 + digito;

Si dato > 4 Entonces

valido <- Falso;

FinSi;

FinSi;

posicion <- posicion + 1;

FinMientras;

Si dato < 1 Entonces

valido <- Falso;

FinSi;

Si NO valido Entonces

Escribir "Entrada invalida. Intente otra vez.";

FinSi;

Hasta Que valido;

producto <- dato;

Repetir

Escribir "Cantidad entera positiva:";

Leer texto;

valido <- Longitud(texto) > 0;

dato <- 0;

posicion <- 0;

Mientras posicion < Longitud(texto) Y valido Hacer

caracter <- Subcadena(texto, posicion, posicion);

digito <- -1;

Para d <- 0 Hasta 9 Hacer

Si caracter = ConvertirATexto(d) Entonces

digito <- d;

FinSi;

FinPara;

Si digito = -1 Entonces

valido <- Falso;

SiNo

dato <- dato * 10 + digito;

Si dato > 2147483647 Entonces

valido <- Falso;

FinSi;

FinSi;

posicion <- posicion + 1;

```

FinMientras;
Si dato < 1 Entonces
    valido <- Falso;
FinSi;
Si NO valido Entonces
    Escribir "Entrada invalida. Intente otra vez.";
FinSi;
Hasta Que valido;
cantidad <- dato;
Segun producto Hacer
1:
    precio <- 1.00;
    cafe <- cafe + cantidad;
2:
    precio <- 2.50;
    sandwich <- sandwich + cantidad;
3:
    precio <- 1.50;
    jugo <- jugo + cantidad;
4:
    precio <- 1.25;
    empanada <- empanada + cantidad;
FinSegun;
importe <- precio * cantidad;
ventas <- ventas + 1;
unidades <- unidades + cantidad;
total <- total + importe;
Escribir "Venta: $", importe;
2:
Escribir "Numero de ventas: ", ventas;
Escribir "Cantidad total de productos: ", unidades;
Escribir "Total recaudado: $", total;
Si ventas = 0 Entonces
    Escribir "Sin ventas: promedio no disponible; sin producto lider.";
SiNo
    promedio <- total / ventas;
    Escribir "Promedio por venta: $", promedio;
    mayor <- cafe;
    Si sandwich > mayor Entonces
        mayor <- sandwich;
    FinSi;
    Si jugo > mayor Entonces
        mayor <- jugo;
    FinSi;
    Si empanada > mayor Entonces
        mayor <- empanada;
    FinSi;
    Escribir "Mayor cantidad vendida: ", mayor;
    Si cafe = mayor Entonces
        Escribir "Cafe";
    FinSi;
    Si sandwich = mayor Entonces
        Escribir "Sandwich";
    FinSi;
    Si jugo = mayor Entonces

```

```
        Escribir "Jugo";
    FinSi;
    Si empanada = mayor Entonces
        Escribir "Empanada";
    FinSi;
FinSi;
3:
    Escribir "1. Cafe: $1.00";
    Escribir "2. Sandwich: $2.50";
    Escribir "3. Jugo: $1.50";
    Escribir "4. Empanada: $1.25";
4:
    Escribir "Programa finalizado.";
FinSegun;
Hasta Que opcion = 4;
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Cada caso comienza con todos los contadores y acumuladores en cero. Las entradas inválidas se corrigen antes de continuar y no cuentan como ventas.

Caso 1 — Sin ventas

Variable	Valor
Opciones	3, 2, 4
ventas / unidades	0 / 0
cafe / sandwich / jugo / empanada	0 / 0 / 0 / 0
total	\$0.00
promedio y líder	No disponibles

Operaciones:

Se consulta la tabla de productos y luego las estadísticas.

No se evalúa 0 / 0 porque ventas = 0. La consulta no altera los contadores.

Caso 2 — Cuatro productos y líder único

Variable	Valor
Registros (producto, cantidad)	(1, 2), (2, 1), (3, 3), (4, 4)
Opciones	1, 1, 1, 1, 2, 4
ventas / unidades	4 / 10
cafe / sandwich / jugo / empanada	2 / 1 / 3 / 4
total / promedio	\$14.00 / \$3.50
mayor / líder	4 / Empanada

Operaciones:

Café: $1.00 \times 2 = 2.00$; total = 2.00; ventas = 1; unidades = 2.

Sándwich: $2.50 \times 1 = 2.50$; total = 4.50; ventas = 2; unidades = 3.

Jugo: $1.50 \times 3 = 4.50$; total = 9.00; ventas = 3; unidades = 6.

Empanada: $1.25 \times 4 = 5.00$; total = 14.00; ventas = 4; unidades = 10.

Promedio = $14.00 / 4 = 3.50$. Mayor: $2 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

Caso 3 — Validaciones y empate

Variable	Valor
Opción intentada	abc, 7 $\rightarrow$ 1
Producto intentado	0, 5 $\rightarrow$ 1
Cantidad intentada	0, -2, 1.5, abc $\rightarrow$ 2
Segunda venta	Producto 3, cantidad 2
Opciones finales	2, 4
ventas / unidades	2 / 4
cafe / sandwich / jugo / empanada	2 / 0 / 2 / 0
total / promedio	\$5.00 / \$2.50
mayor / líderes	2 / Café y Jugo

Operaciones:

Las entradas inválidas se rechazan sin actualizar ningún contador.

Primera venta: $1.00 \times 2 = 2.00$; total = 2.00; ventas = 1.

Segunda venta: $1.50 \times 2 = 3.00$; total = 2.00 + 3.00 = 5.00.

Unidades = 2 + 2 = 4. Promedio = $5.00 / 2 = 2.50$.

Café = Jugo = 2; las condiciones independientes muestran ambos.

Resumen de los casos

Caso	Ventas	Unidades	Total	Promedio	Líder
1	0	0	\$0.00	No aplica	Ninguno
2	4	10	\$14.00	\$3.50	Empanada
3	2	4	\$5.00	\$2.50	Café y Jugo

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class ControlVentasCafeteria {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int opcion, producto, cantidad, posicion;
        char caracter;
        String texto;
        boolean valido;
        double dato, precio = 0, importe, total = 0, promedio;
        long ventas = 0, unidades = 0, mayor;
        long cafe = 0, sandwich = 0, jugo = 0, empanada = 0;
        do {
            System.out.println("===== CAFETERIA UNIVERSITARIA =====");
            System.out.println("1. Registrar venta");
            System.out.println("2. Mostrar estadísticas");
            System.out.println("3. Mostrar tabla de productos");
            System.out.println("4. Salir");
            do {
                System.out.println("Opcion (1 a 4):");
                texto = entrada.nextLine();
                valido = texto.length() > 0;
                dato = 0;
                posicion = 0;
                // Solo se aceptan digitos y un valor dentro del rango.
                while (posicion < texto.length() && valido) {
                    caracter = texto.charAt(posicion);
                    if (caracter < '0' || caracter > '9') {
                        valido = false;
                    } else {
                        dato = dato * 10 + (caracter - '0');
                        if (dato > 4) valido = false;
                    }
                    posicion++;
                }
                if (dato < 1) valido = false;
                if (!valido) System.out.println("Entrada invalida. Intente otra vez.");
            } while (!valido);
            opcion = (int) dato;
            switch (opcion) {
                case 1:
                    do {
                        System.out.println("Producto: 1 Cafe, 2 Sandwich, 3 Jugo, 4 Empanada");
                        texto = entrada.nextLine();
                        valido = texto.length() > 0;
                        dato = 0;
                        posicion = 0;
                        // Solo se aceptan digitos y un valor dentro del rango.
                        while (posicion < texto.length() && valido) {
                            caracter = texto.charAt(posicion);
                            if (caracter < '0' || caracter > '9') {
```

```

        valido = false;
    } else {
        dato = dato * 10 + (caracter - '0');
        if (dato > 4) valido = false;
    }
    posicion++;
}
if (dato < 1) valido = false;
if (!valido) System.out.println("Entrada invalida. Intente otra vez.");
} while (!valido);
producto = (int) dato;
do {
    System.out.println("Cantidad entera positiva:");
    texto = entrada.nextLine();
    valido = texto.length() > 0;
    dato = 0;
    posicion = 0;
    // Solo se aceptan digitos y un valor dentro del rango.
    while (posicion < texto.length() && valido) {
        caracter = texto.charAt(posicion);
        if (caracter < '0' || caracter > '9') {
            valido = false;
        } else {
            dato = dato * 10 + (caracter - '0');
            if (dato > 2147483647) valido = false;
        }
        posicion++;
    }
    if (dato < 1) valido = false;
    if (!valido) System.out.println("Entrada invalida. Intente otra vez.");
} while (!valido);
cantidad = (int) dato;
// Cada registro valido cuenta como una venta.
switch (producto) {
    case 1:
        precio = 1.00;
        cafe += cantidad;
        break;
    case 2:
        precio = 2.50;
        sandwich += cantidad;
        break;
    case 3:
        precio = 1.50;
        jugo += cantidad;
        break;
    case 4:
        precio = 1.25;
        empanada += cantidad;
        break;
}
importe = precio * cantidad;
ventas++;
unidades += cantidad;
total += importe;

```

```

        System.out.printf("Venta: $%.2f%n", importe);
        break;
    case 2:
        System.out.println("Numero de ventas: " + ventas);
        System.out.println("Cantidad total de productos: " + unidades);
        System.out.printf("Total recaudado: $%.2f%n", total);
        if (ventas == 0) {
            System.out.println("Sin ventas: promedio no disponible; sin producto lider.");
        } else {
            promedio = total / ventas;
            System.out.printf("Promedio por venta: $%.2f%n", promedio);
            mayor = cafe;
            if (sandwich > mayor) mayor = sandwich;
            if (jugo > mayor) mayor = jugo;
            if (empanada > mayor) mayor = empanada;
            System.out.println("Mayor cantidad vendida: " + mayor);
            // Condiciones separadas para mostrar los empates.
            if (cafe == mayor) System.out.println("Cafe");
            if (sandwich == mayor) System.out.println("Sandwich");
            if (jugo == mayor) System.out.println("Jugo");
            if (empanada == mayor) System.out.println("Empanada");
        }
        break;
    case 3:
        System.out.println("1. Cafe: $1.00");
        System.out.println("2. Sandwich: $2.50");
        System.out.println("3. Jugo: $1.50");
        System.out.println("4. Empanada: $1.25");
        break;
    case 4:
        System.out.println("Programa finalizado.");
        break;
    }
} while (opcion != 4);
entrada.close();
}
}

```

C++

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    cout << fixed << setprecision(2);
    int opcion, producto, cantidad, posicion;
    char caracter;
    string texto;
    bool valido;
    double dato, precio = 0, importe, total = 0, promedio;
    long long ventas = 0, unidades = 0, mayor;
    long long cafe = 0, sandwich = 0, jugo = 0, empanada = 0;
    do {
        cout << "===== CAFETERIA UNIVERSITARIA =====" << "\n";
        cout << "1. Registrar venta" << "\n";
        cout << "2. Mostrar estadísticas" << "\n";
        cout << "3. Mostrar tabla de productos" << "\n";
        cout << "4. Salir" << "\n";
        do {
            cout << "Opcion (1 a 4):\n";
            getline(cin, texto);
            valido = texto.size() > 0;
            dato = 0;
            posicion = 0;
            // Solo se aceptan digitos y un valor dentro del rango.
            while (posicion < texto.size() && valido) {
                caracter = texto[posicion];
                if (caracter < '0' || caracter > '9') {
                    valido = false;
                } else {
                    dato = dato * 10 + (caracter - '0');
                    if (dato > 4) valido = false;
                }
                posicion++;
            }
            if (dato < 1) valido = false;
            if (!valido) cout << "Entrada invalida. Intente otra vez.\n";
        } while (!valido);
        opcion = (int) dato;
        switch (opcion) {
            case 1:
                do {
                    cout << "Producto: 1 Cafe, 2 Sandwich, 3 Jugo, 4 Empanada\n";
                    getline(cin, texto);
                    valido = texto.size() > 0;
                    dato = 0;
                    posicion = 0;
                    // Solo se aceptan digitos y un valor dentro del rango.
                    while (posicion < texto.size() && valido) {
                        caracter = texto[posicion];
                        if (caracter < '0' || caracter > '9') {
```

```

        valido = false;
    } else {
        dato = dato * 10 + (caracter - '0');
        if (dato > 4) valido = false;
    }
    posicion++;
}
if (dato < 1) valido = false;
if (!valido) cout << "Entrada invalida. Intente otra vez.\n";
} while (!valido);
producto = (int) dato;
do {
    cout << "Cantidad entera positiva:\n";
    getline(cin, texto);
    valido = texto.size() > 0;
    dato = 0;
    posicion = 0;
    // Solo se aceptan digitos y un valor dentro del rango.
    while (posicion < texto.size() && valido) {
        caracter = texto[posicion];
        if (caracter < '0' || caracter > '9') {
            valido = false;
        } else {
            dato = dato * 10 + (caracter - '0');
            if (dato > 2147483647) valido = false;
        }
        posicion++;
    }
    if (dato < 1) valido = false;
    if (!valido) cout << "Entrada invalida. Intente otra vez.\n";
} while (!valido);
cantidad = (int) dato;
// Cada registro valido cuenta como una venta.
switch (producto) {
    case 1:
        precio = 1.00;
        cafe += cantidad;
        break;
    case 2:
        precio = 2.50;
        sandwich += cantidad;
        break;
    case 3:
        precio = 1.50;
        jugo += cantidad;
        break;
    case 4:
        precio = 1.25;
        empanada += cantidad;
        break;
}
importe = precio * cantidad;
ventas++;
unidades += cantidad;
total += importe;

```

```

    cout << "Venta: $" << importe << "\n";
    break;
case 2:
    cout << "Numero de ventas: " << ventas << "\n";
    cout << "Cantidad total de productos: " << unidades << "\n";
    cout << "Total recaudado: $" << total << "\n";
    if (ventas == 0) {
        cout << "Sin ventas: promedio no disponible; sin producto lider." << "\n";
    } else {
        promedio = total / ventas;
        cout << "Promedio por venta: $" << promedio << "\n";
        mayor = cafe;
        if (sandwich > mayor) mayor = sandwich;
        if (jugo > mayor) mayor = jugo;
        if (empanada > mayor) mayor = empanada;
        cout << "Mayor cantidad vendida: " << mayor << "\n";
        // Condiciones separadas para mostrar los empates.
        if (cafe == mayor) cout << "Cafe" << "\n";
        if (sandwich == mayor) cout << "Sandwich" << "\n";
        if (jugo == mayor) cout << "Jugo" << "\n";
        if (empanada == mayor) cout << "Empanada" << "\n";
    }
    break;
case 3:
    cout << "1. Cafe: $1.00" << "\n";
    cout << "2. Sandwich: $2.50" << "\n";
    cout << "3. Jugo: $1.50" << "\n";
    cout << "4. Empanada: $1.25" << "\n";
    break;
case 4:
    cout << "Programa finalizado." << "\n";
    break;
}
} while (opcion != 4);
return 0;
}

```

DETECTOR Y DEPURADOR DE ERRORES

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Analice el siguiente código:

```
int numero = 1;
```

```
while (numero <= 10) {  
    System.out.println(numero);  
}
```

El estudiante deberá:

1. Identificar el problema.
2. Explicar por qué ocurre.
3. Corregirlo.
4. Elaborar una tabla de trazas.
5. Demostrar la solución ejecutándose correctamente.

Posteriormente deberá crear voluntariamente otro ejemplo que produzca un error relacionado con ciclos y corregirlo.

En el README.md deberá documentar:

ERROR ENCONTRADO:

CAUSA:

SOLUCIÓN:

RESULTADO:

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- El usuario no ingresa datos. El enunciado establece $\text{numero} = 1$ y el límite 10, ambos enteros.
- Para el ejemplo adicional se define $\text{contador} = 5$ y el límite inferior 1, ambos enteros. Se busca imprimir 5, 4, 3, 2 y 1.

1.2 Proceso

- Declarar numero y contador como enteros.
- Asignar 1 a numero y mostrar el encabezado del primer ejemplo.
- Evaluar $\text{numero} \leq 10$ antes de cada repetición. Si es verdadero, mostrar numero y aumentar su valor en 1. Si es falso, continuar con el segundo ejemplo.
- Asignar 5 a contador y mostrar el encabezado del segundo ejemplo.
- Evaluar $\text{contador} \geq 1$. Si es verdadero, mostrar contador y disminuir su valor en 1. Si es falso, terminar.
- Las condiciones controlan los límites y la finalización. No se necesita menú ni validación de entradas porque no se solicitan datos.

1.3 Salida de datos

- Encabezado “Ejemplo 1: conteo ascendente” y los números del 1 al 10, uno por línea.
- Encabezado “Ejemplo 2: cuenta regresiva” y los números 5, 4, 3, 2 y 1, uno por línea.

2. ALGORITMO

1. Inicio.

2. Declarar numero y contador como enteros.

3. Asignar $\text{numero} = 1$.

4. Mostrar “Ejemplo 1: conteo ascendente”.

5. Si $\text{numero} \leq 10$, seguir al paso 6; en caso contrario, ir al paso 8.

6. Mostrar numero .

7. Sumar 1 a numero y regresar al paso 5.

8. Asignar $\text{contador} = 5$.

9. Mostrar “Ejemplo 2: cuenta regresiva”.

10. Si $\text{contador} \geq 1$, seguir al paso 11; en caso contrario, ir al paso 13.

11. Mostrar contador .

12. Restar 1 a contador y regresar al paso 10.

13. **Fin.**

3. PSEUDOCÓDIGO

Algoritmo DetectorErrores

Definir numero, contador Como Entero;

numero <- 1;

Escribir "Ejemplo 1: conteo ascendente";

Mientras numero <= 10 Hacer

 Escribir numero;

 numero <- numero + 1;

FinMientras

contador <- 5;

Escribir "Ejemplo 2: cuenta regresiva";

Mientras contador >= 1 Hacer

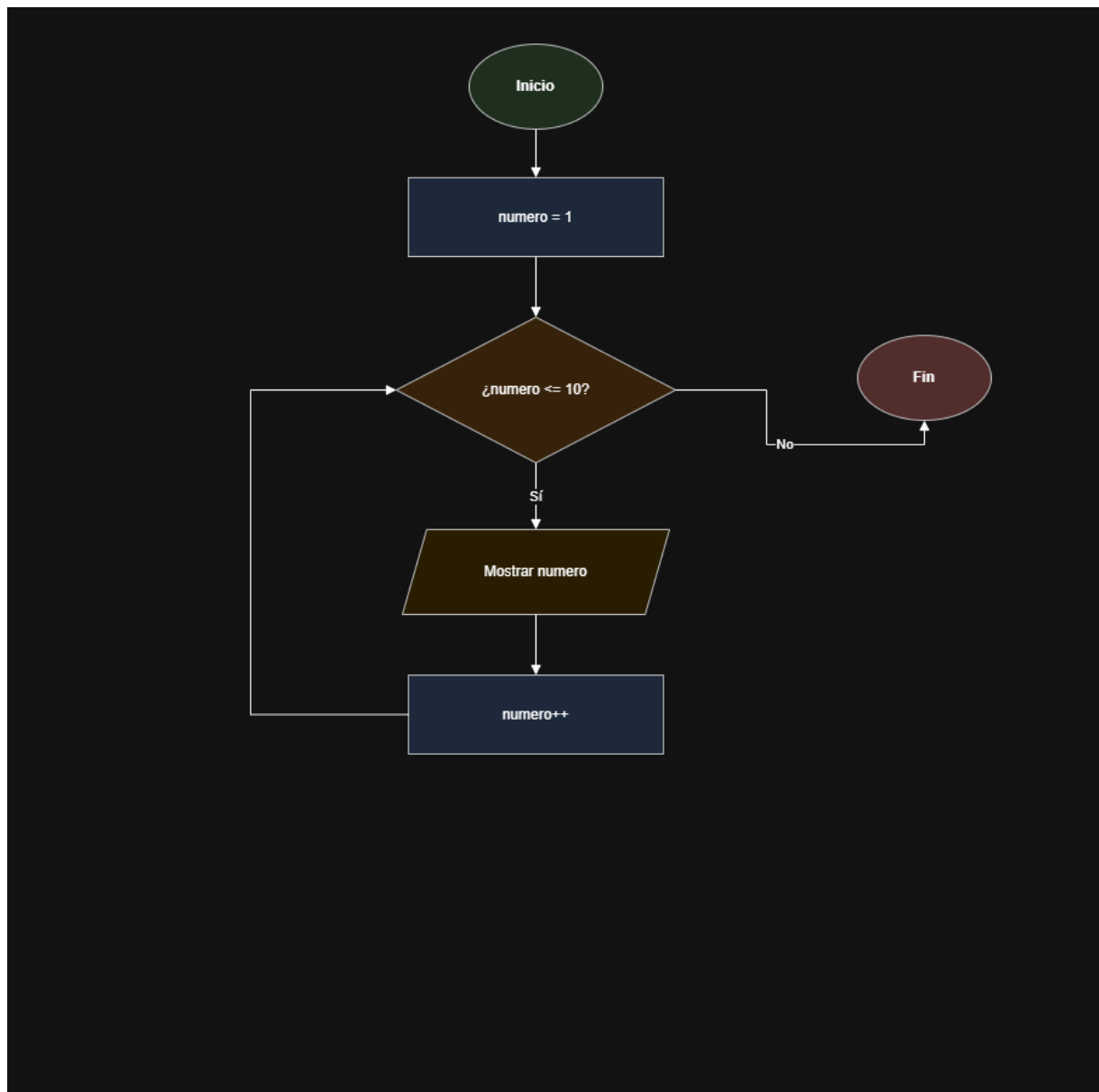
 Escribir contador;

 contador <- contador - 1;

FinMientras

FinAlgoritmo

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1. Código original sin actualización

Se observa el fallo del programa original. La traza se limita a tres repeticiones para representar el patrón; el programa no se detiene por sí solo.

Variable	Valor
Entrada del usuario	Ninguna
numero inicial	1
Límite	10
numero después de cada vuelta	1; no existe una actualización
Condición	$1 \leq 10$: verdadero
Resultado final	No existe terminación natural; imprime 1 indefinidamente

Iteración	numero antes	Condición	Salida	numero después
1	1	Verdadero	1	1
2	1	Verdadero	1	1
3	1	Verdadero	1	1

Paso a paso: se asigna $\text{numero} = 1$; se compara $1 \leq 10$; se imprime 1. No hay suma ni resta, de modo que numero continúa en 1. La siguiente comparación vuelve a ser verdadera.

Caso 2. Conteo ascendente corregido

Se verifica que el primer ciclo incluya el 10 y termine al llegar a 11.

Variable	Valor
Entrada del usuario	Ninguna
numero inicial / límite	1 / 10
Valores intermedios de numero	2, 3, 4, 5, 6, 7, 8, 9 y 10
numero al salir	11
Resultado final	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
Cantidad de impresiones	$10 - 1 + 1 = 10$

Caso 2. Tabla de trazas y operaciones

Evaluación	numero	numero <= 10	Salida	Actualización
1	1	Verdadero	1	$1 + 1 = 2$
2	2	Verdadero	2	$2 + 1 = 3$
3	3	Verdadero	3	$3 + 1 = 4$
4	4	Verdadero	4	$4 + 1 = 5$
5	5	Verdadero	5	$5 + 1 = 6$
6	6	Verdadero	6	$6 + 1 = 7$
7	7	Verdadero	7	$7 + 1 = 8$
8	8	Verdadero	8	$8 + 1 = 9$
9	9	Verdadero	9	$9 + 1 = 10$
10	10	Verdadero	10	$10 + 1 = 11$
11	11	Falso	No imprime	No se ejecuta

Cada fila desarrolla la suma realizada. Tras imprimir 10, se calcula $10 + 1 = 11$. Como $11 \leq 10$ es falso, se abandona el ciclo y comienza el ejemplo adicional.

Caso 3. Cuenta regresiva corregida

Se verifica la inclusión del límite inferior 1, que el ejemplo adicional erróneo omitía.

Variable	Valor
Entrada del usuario	Ninguna
contador inicial / límite	5 / 1
Valores intermedios de contador	4, 3, 2 y 1
contador al salir	0
Resultado final	5, 4, 3, 2, 1
Cantidad de impresiones	$5 - 1 + 1 = 5$

contador	contador >= 1	Salida	Actualización
5	Verdadero	5	$5 - 1 = 4$
4	Verdadero	4	$4 - 1 = 3$
3	Verdadero	3	$3 - 1 = 2$
2	Verdadero	2	$2 - 1 = 1$
1	Verdadero	1	$1 - 1 = 0$
0	Falso	No imprime	No se ejecuta

Caso 3. Comparación con el error

Con contador > 1 , al llegar a 1 se evalúa $1 > 1$, que es falso: se omite el 1 y solo hay $5 - 2 + 1 = 4$ impresiones. Con contador ≥ 1 , se evalúa $1 \geq 1$, que es verdadero: se imprime 1, se calcula $1 - 1 = 0$ y la siguiente condición es falsa. No se necesitan redondeos porque todas las operaciones son enteras.

Resumen de los casos

Caso	Comportamiento comprobado	Resultado
1. Original	Falta la actualización	Repite 1 indefinidamente
2. Corregido	Incluye 10 y sale en 11	10 impresiones
3. Adicional corregido	Incluye 1 y sale en 0	5 impresiones

6. CÓDIGO

JAVA

```
public class DetectorErrores {  
    public static void main(String[] args) {  
        int numero;  
        int contador;  
  
        // Primer ciclo: mostrar los numeros del 1 al 10.  
        numero = 1;  
        System.out.println("Ejemplo 1: conteo ascendente");  
        while (numero <= 10) {  
            System.out.println(numero);  
            numero++; // Avanzamos para que el ciclo termine.  
        }  
  
        // Segundo ciclo: incluir el 1 en la cuenta regresiva.  
        contador = 5;  
        System.out.println("Ejemplo 2: cuenta regresiva");  
        while (contador >= 1) {  
            System.out.println(contador);  
            contador--;  
        }  
    }  
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    int numero;
    int contador;

    // Primer ciclo: mostrar los numeros del 1 al 10.
    numero = 1;
    cout << "Ejemplo 1: conteo ascendente" << endl;
    while (numero <= 10) {
        cout << numero << endl;
        numero++; // Avanzamos para que el ciclo termine.
    }

    // Segundo ciclo: incluir el 1 en la cuenta regresiva.
    contador = 5;
    cout << "Ejemplo 2: cuenta regresiva" << endl;
    while (contador >= 1) {
        cout << contador << endl;
        contador--;
    }
    return 0;
}
```

ESTADÍSTICAS DE UNA ENCUESTA UNIVERSITARIA

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Pregunte inicialmente cuántos estudiantes participarán. Por cada estudiante solicitar:

- Edad.
- Semestre.
- Horas de estudio por día.

Validaciones:

- Edad: 16 a 80.
- Semestre: 1 a 10.
- Horas de estudio: 0 a 24.

Determinar:

- Edad promedio.
- Horas promedio de estudio.
- Estudiante con mayor cantidad de horas.
- Estudiantes que estudian menos de 2 horas.
- Cantidad de estudiantes por semestre.

Para el último requisito deberán utilizar ciclos anidados o una estrategia equivalente explicada por el estudiante.

Criterios de implementación

Cada estudiante se identifica por su orden de registro: 1, 2, 3, etc. En un empate de horas se conserva al primero registrado. Se admite una cantidad positiva de participantes y se aceptan horas decimales con punto. Para “estudiantes que estudian menos de 2 horas” se informa la cantidad de estudiantes que cumplen la condición.

Se emplea un arreglo de contadores por semestre como estrategia equivalente a los ciclos anidados. No se agrega un menú porque el ejercicio consiste en realizar una encuesta y presentar sus resultados.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- n: cantidad de participantes; entero mayor que cero. La implementación usa enteros de 32 bits: de 1 a 2147483647. Ese máximo es un límite técnico del tipo de dato.
- edad: número entero entre 16 y 80, incluidos ambos límites.
- semestre: número entero entre 1 y 10, incluidos ambos límites.
- horas: número real entre 0 y 24, incluidos ambos límites; puede contener decimales.
- Se escribe un número por línea, sin espacios ni separadores de miles; se usa punto decimal. Texto vacío, letras, signos negativos, formatos incorrectos y valores fuera del rango se rechazan.

1.2 Proceso

- Solicitar n mediante LeerDato y repetir su lectura hasta que sea un entero válido y positivo.
- Inicializar sumaEdades, sumaHoras, menosDos y estudianteMayor en 0; mayorHoras en -1. Recorrer los semestres 1 a 10 para poner sus contadores en 0.
- Recorrer los n estudiantes. Mostrar su número y solicitar edad, semestre y horas; validar cada dato antes de continuar.
- Acumular edad en sumaEdades y horas en sumaHoras. Aumentar cantidad[semestre] en 1.
- Si horas > mayorHoras, actualizar mayorHoras y guardar el número del estudiante. El signo > conserva al primero cuando hay empate.
- Si horas < 2, aumentar menosDos en 1. Exactamente 2 horas no cumple esta condición.
- Calcular promedioEdad = sumaEdades / n y promedioHoras = sumaHoras / n. n es positivo, por lo que no hay división para cero.
- Mostrar los resultados y recorrer los diez semestres para presentar sus cantidades, incluidos los ceros. Solo redondear los resultados al mostrarlos.

1.3 Salida de datos

- Solicitudes de datos, número del estudiante en registro y mensajes de error cuando la entrada no es válida.
- Edad promedio y horas promedio de estudio, presentadas con dos decimales en Java y C++.
- Número del estudiante con más horas y su cantidad de horas diarias.
- Cantidad de estudiantes que estudian menos de 2 horas al día.
- Cantidad de estudiantes de cada semestre, del 1 al 10.

2. ALGORITMO

1. Inicio.

2. Declarar las variables de entrada, los acumuladores, los contadores, los promedios y el arreglo cantidad de 11 posiciones; se usarán las posiciones 1 a 10.
3. Solicitar n. Si el formato no es numérico, no es entero o está fuera del rango permitido, mostrar un error y repetir la lectura.
4. Inicializar sumaEdades = 0, sumaHoras = 0, menosDos = 0, mayorHoras = -1 y estudianteMayor = 0.
5. Para cada semestre s desde 1 hasta 10, asignar cantidad[s] = 0.
6. Asignar i = 0. Si i >= n, continuar en el paso 14.
7. Mostrar el número del estudiante i + 1.
8. Solicitar edad y repetir su lectura hasta obtener un entero de 16 a 80.
9. Solicitar semestre y repetir su lectura hasta obtener un entero de 1 a 10.
10. Solicitar horas y repetir su lectura hasta obtener un número de 0 a 24.
11. Sumar edad a sumaEdades y horas a sumaHoras; aumentar cantidad[semestre] en 1.
12. Si horas > mayorHoras, asignar mayorHoras = horas y estudianteMayor = i + 1. Si horas < 2, aumentar menosDos en 1.
13. Aumentar i en 1 y regresar a la comprobación del paso 6, sin reinicializar i.
14. Calcular promedioEdad = sumaEdades / n y promedioHoras = sumaHoras / n.
15. Mostrar ambos promedios, el estudiante con más horas, sus horas y la cantidad que estudia menos de 2 horas.
16. Recorrer s desde 1 hasta 10 y mostrar s y cantidad[s].
17. Fin.

3. PSEUDOCÓDIGO

Subproceso de lectura y validación

La lectura se hace como texto para detectar letras o formatos incorrectos. Se admite una parte entera y, opcionalmente, un punto seguido de dígitos. Después se verifican el rango y la condición de número entero.

SubProceso valor <- LeerDato(mensaje, minimo, maximo, entero)

Definir mensaje, texto, caracter Como Cadena;

Definir minimo, maximo, valor, divisor Como Real;

Definir entero, valido, decimal, digito Como Logico;

Definir k Como Entero;

Repetir

 Escribir Sin Saltar mensaje;

 Leer texto;

 valor <- 0;

 divisor <- 1;

 decimal <- Falso;

 valido <- Longitud(texto) > 0;

 k <- 0;

 Mientras k < Longitud(texto) Y valido Hacer

 caracter <- Subcadena(texto, k, k);

 digito <- caracter = "0" O caracter = "1";

 digito <- digito O caracter = "2" O caracter = "3";

 digito <- digito O caracter = "4" O caracter = "5";

 digito <- digito O caracter = "6" O caracter = "7";

 digito <- digito O caracter = "8" O caracter = "9";

 Si digito Entonces

 valor <- valor * 10 + ConvertirANumero(caracter);

 Si decimal Entonces

 divisor <- divisor * 10;

 FinSi

 SiNo

 Si caracter = "." Y No decimal Entonces

 Si k = 0 O k = Longitud(texto) - 1 Entonces

 valido <- Falso;

 FinSi

Subproceso de lectura y validación — continuación

```
        decimal <- Verdadero;
    SiNo
        valido <- Falso;
    FinSi
    FinSi
    k <- k + 1;
    FinMientras
    valor <- valor / divisor;
    valido <- valido Y valor >= minimo Y valor <= maximo;
    Si entero Y valor <> Trunc(valor) Entonces
        valido <- Falso;
    FinSi
    Si No valido Entonces
        Escribir "Dato invalido. Intente nuevamente.";
    FinSi
    Hasta Que valido
FinSubProceso
```

Ejemplo de conversión: para “1.5”, se forma valor = 15 y divisor = 10; al finalizar, $15 / 10 = 1,5$. Si se solicitan enteros, el dato se acepta solamente cuando valor = Trunc(valor).

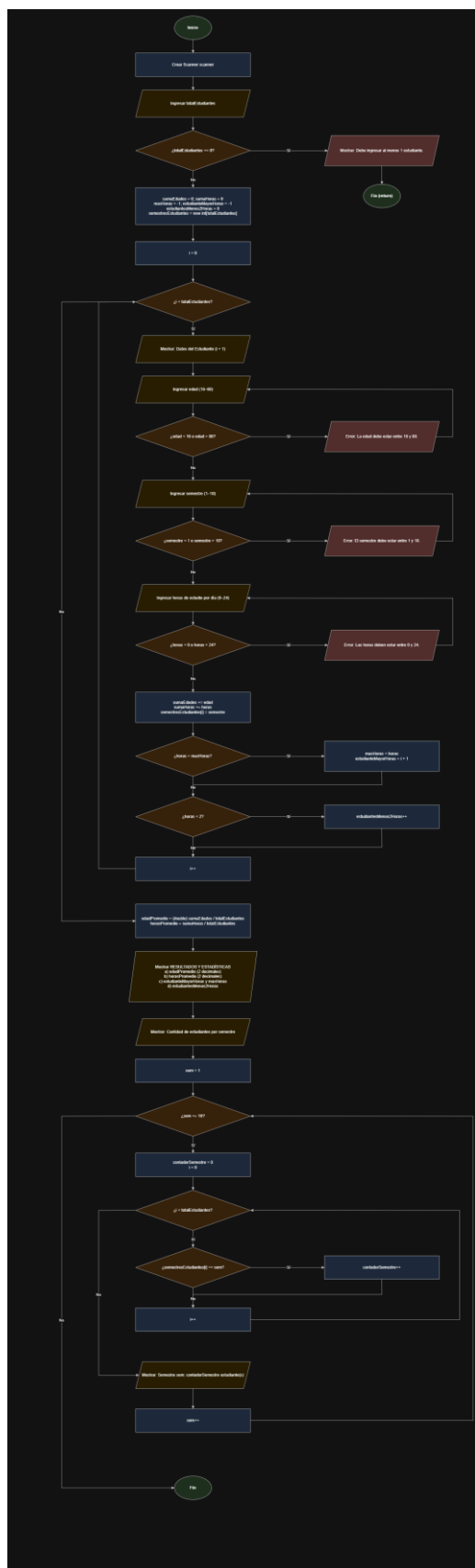
Algoritmo principal

```
Algoritmo EncuestaUniversitaria
    Definir n, edad, semestre, i, s Como Entero;
    Definir estudianteMayor, menosDos Como Entero;
    Definir cantidad Como Entero;
    Dimension cantidad[11];
    Definir horas, sumaEdades, sumaHoras Como Real;
    Definir mayorHoras, promedioEdad, promedioHoras Como Real;
    n <- LeerDato("Cantidad de estudiantes: ", 1, 2147483647, Verdadero);
    sumaEdades <- 0;
    sumaHoras <- 0;
    mayorHoras <- -1;
    estudianteMayor <- 0;
    menosDos <- 0;
    Para s <- 1 Hasta 10 Hacer
        cantidad[s] <- 0;
    FinPara
```

Algoritmo principal — continuación

```
Para i <- 0 Hasta n - 1 Hacer
  Escribir "Estudiante ", i + 1;
  edad <- LeerDato("Edad: ", 16, 80, Verdadero);
  semestre <- LeerDato("Semestre: ", 1, 10, Verdadero);
  horas <- LeerDato("Horas por día: ", 0, 24, Falso);
  sumaEdades <- sumaEdades + edad;
  sumaHoras <- sumaHoras + horas;
  cantidad[semestre] <- cantidad[semestre] + 1;
  Si horas > mayorHoras Entonces
    mayorHoras <- horas;
    estudianteMayor <- i + 1;
  FinSi
  Si horas < 2 Entonces
    menosDos <- menosDos + 1;
  FinSi
FinPara
promedioEdad <- sumaEdades / n;
promedioHoras <- sumaHoras / n;
Escribir "Edad promedio: ", Redon(promedioEdad * 100) / 100;
Escribir "Horas promedio: ", Redon(promedioHoras * 100) / 100;
Escribir "Estudiante con mas horas: ", estudianteMayor;
Escribir "Mayor cantidad de horas: ", Redon(mayorHoras * 100) / 100;
Escribir "Estudiantes con menos de 2 horas: ", menosDos;
Para s <- 1 Hasta 10 Hacer
  Escribir "Semestre ", s, ": ", cantidad[s];
FinPara
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1. Datos variados y repetición de semestre

Tres estudiantes permiten verificar los promedios, el máximo, la condición de menos de 2 horas y dos registros en un mismo semestre.

Variable	Valor
n	3
edad, semestre, horas de E1	18; 1; 1,5
edad, semestre, horas de E2	20; 3; 4
edad, semestre, horas de E3	22; 3; 2
sumaEdades / sumaHoras finales	60 / 7,5
promedioEdad / promedioHoras	20,00 / 2,50
estudianteMayor / mayorHoras	2 / 4,00
menosDos	1
cantidad[1..10]	[1, 0, 2, 0, 0, 0, 0, 0, 0, 0]

Registro	sumaEdades	sumaHoras	Mayor (est.)	menosDos
Inicial	0	0	-1 (0)	0
E1	18	1,5	1,5 (1)	1
E2	38	5,5	4 (2)	1
E3	60	7,5	4 (2)	1

Operaciones paso a paso

E1: $0 + 18 = 18$; $0 + 1,5 = 1,5$; $\text{cantidad}[1] = 0 + 1 = 1$. Como $1,5 > -1$, el máximo pasa a E1; $1,5 < 2$ aumenta menosDos de 0 a 1.

E2: $18 + 20 = 38$; $1,5 + 4 = 5,5$; $\text{cantidad}[3] = 0 + 1 = 1$. Como $4 > 1,5$, el máximo pasa a E2; $4 < 2$ es falso.

E3: $38 + 22 = 60$; $5,5 + 2 = 7,5$; $\text{cantidad}[3] = 1 + 1 = 2$. Como $2 > 4$ y $2 < 2$ son falsos, el máximo y menosDos se mantienen.

Edad promedio = $60 / 3 = 20$. Horas promedio = $7,5 / 3 = 2,5$. Comprobación: $1 + 2 = 3$ estudiantes en los semestres.

Caso 2. Límites inferiores y superiores

Se verifican las edades 16 y 80, los semestres 1 y 10 y las horas 0 y 24. Todos los límites son válidos.

Variable	Valor
n	2
edad, semestre, horas de E1	16; 1; 0
edad, semestre, horas de E2	80; 10; 24
sumaEdades / sumaHoras finales	96 / 24
promedioEdad / promedioHoras	48,00 / 12,00
estudianteMayor / mayorHoras	2 / 24,00
menosDos	1
cantidad[1..10]	[1, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Registro	sumaEdades	sumaHoras	Mayor (est.)	menosDos
Inicial	0	0	-1 (0)	0
E1	16	0	0 (1)	1
E2	96	24	24 (2)	1

Operaciones paso a paso

E1: $0 + 16 = 16$; $0 + 0 = 0$; $\text{cantidad}[1] = 0 + 1 = 1$. Como $0 > -1$, E1 queda registrado como máximo inicial; $0 < 2$ aumenta menosDos a 1.

E2: $16 + 80 = 96$; $0 + 24 = 24$; $\text{cantidad}[10] = 0 + 1 = 1$. Como $24 > 0$, el máximo pasa a E2; $24 < 2$ es falso.

Edad promedio = $96 / 2 = 48$. Horas promedio = $24 / 2 = 12$. Comprobación: $1 + 1 = 2$ estudiantes en los semestres.

Caso 3. Entradas inválidas, empate y exactamente 2 horas

Se repiten las lecturas incorrectas. Los dos estudiantes válidos estudian 2 horas: se conserva al primero como máximo y ninguno pertenece al grupo de menos de 2 horas.

Variable	Valor
n intentado → válido	0 (inválido) → 2
edad E1 intentada → válida	15, 81 (inválidas) → 18
semestre E1 intentado → válido	0, 11 (inválidos) → 2
horas E1 intentadas → válidas	abc, -1, 25 (inválidas) → 2
edad, semestre, horas de E2	20; 2; 2
sumaEdades / sumaHoras finales	38 / 4
promedioEdad / promedioHoras	19,00 / 2,00
estudianteMayor / mayorHoras	1 / 2,00
menosDos / errores de entrada	0 / 8
cantidad[1..10]	[0, 2, 0, 0, 0, 0, 0, 0, 0, 0]

Operaciones paso a paso

Los ocho datos inválidos generan mensajes de error. No se suman, no aumentan los contadores y no consumen un registro de estudiante.

E1 válido: $0 + 18 = 18$; $0 + 2 = 2$; $\text{cantidad}[2] = 0 + 1 = 1$. Como $2 > -1$, el máximo pasa a E1. Como $2 < 2$ es falso, $\text{menosDos} = 0$.

E2: $18 + 20 = 38$; $2 + 2 = 4$; $\text{cantidad}[2] = 1 + 1 = 2$. La comparación $2 > 2$ es falsa: el máximo sigue siendo E1. También es falso $2 < 2$.

Edad promedio = $38 / 2 = 19$. Horas promedio = $4 / 2 = 2$. Comprobación: $\text{cantidad}[2] = 2$, igual a n.

Resumen de los casos

Caso	Edad prom.	Horas prom.	Máximo	Menos de 2 h
1	20,00	2,50	E2: 4 h	1
2	48,00	12,00	E2: 24 h	1
3	19,00	2,00	E1: 2 h	0

6. CÓDIGO

JAVA

Guardar como EncuestaUniversitaria.java. El método leerDato reutiliza la misma validación para las cuatro entradas. Se emplea Scanner y las variables del programa principal se declaran antes del procesamiento.

```
import java.util.Scanner;
import java.util.Locale;

public class EncuestaUniversitaria {
    // Lee un dato y lo repite si no cumple las condiciones.
    static double leerDato(Scanner sc, String mensaje,
        double minimo, double maximo, boolean entero) {
        String texto;
        double valor;
        boolean valido;
        do {
            System.out.print(mensaje);
            texto = sc.nextLine();
            valido = texto.matches("[0-9]+([.][0-9]+)?");
            valor = -1;
            if (valido) {
                valor = Double.parseDouble(texto);
                valido = valor >= minimo && valor <= maximo;
                if (entero && valor % 1 != 0) {
                    valido = false;
                }
            }
            if (!valido) {
                System.out.println("Dato invalido. Intente nuevamente.");
            }
        } while (!valido);
        return valor;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

JAVA — continuación

```
int n, edad, semestre, i, s;
int estudianteMayor, menosDos;
int[] cantidad = new int[11];
double horas, sumaEdades, sumaHoras, mayorHoras;
double promedioEdad, promedioHoras;

n = (int) leerDato(sc, "Cantidad de estudiantes: ",
    1, Integer.MAX_VALUE, true);
sumaEdades = 0;
sumaHoras = 0;
mayorHoras = -1;
estudianteMayor = 0;
menosDos = 0;
for (s = 1; s <= 10; s++) {
    cantidad[s] = 0;
}
// i empieza en cero; el estudiante se identifica con i + 1.
for (i = 0; i < n; i++) {
    System.out.println("Estudiante " + (i + 1));
    edad = (int) leerDato(sc, "Edad: ", 16, 80, true);
    semestre = (int) leerDato(sc, "Semestre: ", 1, 10, true);
    horas = leerDato(sc, "Horas por dia: ", 0, 24, false);
    sumaEdades += edad;
    sumaHoras += horas;
    cantidad[semestre]++;
    if (horas > mayorHoras) {
        mayorHoras = horas;
        estudianteMayor = i + 1;
    }
    if (horas < 2) {
        menosDos++;
    }
}
promedioEdad = sumaEdades / n;
promedioHoras = sumaHoras / n;
System.out.printf(Locale.US, "Edad promedio: %.2f%n", promedioEdad);
System.out.printf(Locale.US, "Horas promedio: %.2f%n", promedioHoras);
System.out.println("Estudiante con mas horas: " + estudianteMayor);
System.out.printf(Locale.US, "Mayor cantidad de horas: %.2f%n", mayorHoras);
System.out.println("Estudiantes con menos de 2 horas: " + menosDos);
for (s = 1; s <= 10; s++) {
    System.out.println("Semestre " + s + ": " + cantidad[s]);
}
sc.close();
}
}
```

C++

Guardar como EncuestaUniversitaria.cpp. Compilar con C++17. Se mantiene la misma entrada, validación, orden de procesamiento, criterio de empate y salida que en Java.

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
#include <regex>
#include <cmath>
#include <limits>
#include <locale>
using namespace std;

// Lee un dato y lo repite si no cumple las condiciones.
double leerDato(string mensaje, double minimo,
                double maximo, bool entero) {
    string texto;
    double valor;
    bool valido;
    do {
        cout << mensaje;
        getline(cin, texto);
        valido = regex_match(texto, regex("[0-9]+([.][0-9]+)?"));
        valor = -1;
        if (valido) {
            istringstream lectura(texto);
            lectura.imbue(locale::classic());
            valido = bool(lectura >> valor);
            valido = valido && valor >= minimo && valor <= maximo;
            if (entero && fmod(valor, 1.0) != 0) {
                valido = false;
            }
        }
    } while (!valido);
    return valor;
}

int main() {
```

C++ — continuación

```
int n, edad, semestre, i, s;
int estudianteMayor, menosDos;
int cantidad[11];
double horas, sumaEdades, sumaHoras, mayorHoras;
double promedioEdad, promedioHoras;

n = int(leerDato("Cantidad de estudiantes: ",
    1, numeric_limits<int>::max(), true));
sumaEdades = 0;
sumaHoras = 0;
mayorHoras = -1;
estudianteMayor = 0;
menosDos = 0;
for (s = 1; s <= 10; s++) {
    cantidad[s] = 0;
}
// i empieza en cero; el estudiante se identifica con i + 1.
for (i = 0; i < n; i++) {
    cout << "Estudiante " << i + 1 << endl;
    edad = int(leerDato("Edad: ", 16, 80, true));
    semestre = int(leerDato("Semestre: ", 1, 10, true));
    horas = leerDato("Horas por dia: ", 0, 24, false);
    sumaEdades += edad;
    sumaHoras += horas;
    cantidad[semestre]++;
    if (horas > mayorHoras) {
        mayorHoras = horas;
        estudianteMayor = i + 1;
    }
    if (horas < 2) {
        menosDos++;
    }
}
promedioEdad = sumaEdades / n;
promedioHoras = sumaHoras / n;
cout.imbue(locale::classic());
cout << fixed << setprecision(2);
cout << "Edad promedio: " << promedioEdad << endl;
cout << "Horas promedio: " << promedioHoras << endl;
cout << "Estudiante con mas horas: " << estudianteMayor << endl;
cout << "Mayor cantidad de horas: " << mayorHoras << endl;
cout << "Estudiantes con menos de 2 horas: " << menosDos << endl;
for (s = 1; s <= 10; s++) {
    cout << "Semestre " << s << ": " << cantidad[s] << endl;
}
return 0;
}
```

Verificación de los programas

La versión C++ se compiló con C++17 y se ejecutó con los tres casos de la prueba de escritorio. Los promedios, el estudiante con más horas, los contadores por semestre y los ocho rechazos del caso 3 coincidieron con los resultados calculados. El código Java se revisó, pero no se ejecutó porque no había un compilador Java disponible en el entorno de revisión.

Comandos para comprobar las versiones:

```
javac EncuestaUniversitaria.java  
java EncuestaUniversitaria
```

```
g++ -std=c++17 EncuestaUniversitaria.cpp -o encuesta  
./encuesta
```

Salida final comprobada del caso 1

```
Edad promedio: 20.00  
Horas promedio: 2.50  
Estudiante con mas horas: 2  
Mayor cantidad de horas: 4.00  
Estudiantes con menos de 2 horas: 1  
Semestre 1: 1  
Semestre 2: 0  
Semestre 3: 2  
Semestre 4: 0  
Semestre 5: 0  
Semestre 6: 0  
Semestre 7: 0  
Semestre 8: 0  
Semestre 9: 0  
Semestre 10: 0
```

La cantidad por semestre debe sumar n. Esta comprobación se cumple en los tres casos y permite detectar registros omitidos o contados más de una vez.

SISTEMA INTEGRADOR DE PARQUEADERO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Construya una aplicación completa utilizando el siguiente menú:

PARQUEADERO UNIVERSITARIO

1. Registrar vehículo
2. Mostrar vehículos registrados
3. Mostrar estadísticas
4. Mostrar recaudación
5. Salir

Por cada vehículo registrar:

- Tipo de vehículo.
- Rol: estudiante, docente o visitante.
- Número de horas.
- Día de la semana.
- Boleto perdido: sí/no.

El programa deberá aplicar diferentes tarifas definidas por el equipo. El reporte final deberá mostrar:

- Vehículos registrados.
- Cantidad por tipo.
- Cantidad por rol.
- Total de horas.
- Promedio de permanencia.
- Total recaudado.
- Mayor valor pagado.
- Menor valor pagado.

Debe integrar obligatoriamente: for + while + do-while + switch + ciclos anidados + contadores + acumuladores + validaciones + casos límite.

Tarifas y criterios definidos para esta solución

El enunciado permite al equipo definir las tarifas. Para este ejercicio se propone la siguiente tabla; son valores académicos, no tarifas oficiales de una universidad.

Concepto	Valor definido
Motocicleta / automóvil / camioneta	\$0,50 / \$1,00 / \$1,50 por hora
Estudiante / docente / visitante	Descuento del 20 % / 10 % / 0 %
Lunes a viernes / sábado / domingo	Factor 1,00 / 1,20 / 1,50
Boleto perdido	Recargo fijo de \$5,00, después del descuento
Permanencia permitida	Horas enteras de 1 a 24
Almacenamiento de la sesión	Máximo 50 registros; no se eliminan registros

El límite de 50 registros es una decisión técnica para trabajar con arreglos estáticos de primer semestre. No representa el número de espacios físicos libres. Los datos se conservan durante la ejecución; al cerrar el programa no se guardan en un archivo.

Orden del cobro

1. Base = tarifa por hora $\times$ horas.
2. Importe por día = base $\times$ factor del día.
3. Importe con descuento = importe por día $\times$ (1 - descuento del rol).
4. Redondear ese importe al centavo más cercano.
5. Si se perdió el boleto, agregar \$5,00.

En los códigos se trabaja con centavos enteros para sumar exactamente los cobros. Se usa factor = 100, 120 o 150 y descuento = 20, 10 o 0. La operación es:

base = tarifaCentavos * horas;

pago = Trunc((base * factor * (100 - descuento) + 5000) / 10000);

Si boleto = 1: pago = pago + 500;

La suma de 5000 permite redondear al centavo antes de truncar. Un empate de medio centavo se redondea hacia arriba. El recargo fijo no recibe descuento. Por ejemplo, $\$0,50 \times 1,50 \times 0,90 = \$0,675$ se cobra como \$0,68.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

- opcion: entero de 1 a 5 según el menú.
- tipo: entero; 1 = motocicleta, 2 = automóvil, 3 = camioneta.
- rol: entero; 1 = estudiante, 2 = docente, 3 = visitante.
- horas: entero de 1 a 24, incluidos los límites.
- dia: entero de 1 a 7; lunes = 1, martes = 2, miércoles = 3, jueves = 4, viernes = 5, sábado = 6 y domingo = 7.
- boleto: entero; 0 = no perdido, 1 = perdido.
- Todos los campos se escriben con dígitos, sin espacios, signos ni decimales. Se rechazan letras, entradas vacías y números fuera del rango.

1.2 Proceso

- Inicializar n, totalHoras, total, mayor y menor en cero. Reservar la matriz datos[50][5] y el arreglo pagos[50].
- Mostrar el menú con do-while; validar la opción con while y seleccionar la acción con switch.
- Para registrar, comprobar que $n < 50$. Si se alcanzó la capacidad, informar y regresar al menú sin leer ni guardar otro vehículo.
- Leer y validar los cinco datos del vehículo. Obtener tarifa, factor y descuento de las tablas; calcular el cobro en centavos y aplicar el recargo cuando corresponda.
- Guardar datos y pago, acumular horas y recaudación; actualizar mayor y menor. En el primer registro, ambos toman el pago recibido. Aumentar n en uno y confirmar el registro.
- En el listado, recorrer solo los n registros con for y mostrar sus datos y pagos. Si $n = 0$, informar que no existen vehículos.
- En estadísticas, usar un for por categoría y un for interno por registro. Contar coincidencias de tipo y de rol; mostrar totalHoras y calcular $\text{totalHoras} / n$ solo si $n > 0$.
- En recaudación, mostrar total, mayor y menor convertidos de centavos a dólares. Sin registros, el total es cero y los extremos no están disponibles.
- Al elegir 5, presentar listado, estadísticas y recaudación como reporte final; después terminar el ciclo y cerrar el programa.

1.3 Salida de datos

- Menú, solicitudes, mensajes de dato inválido y de capacidad alcanzada.
- Confirmación con número de registro y valor a pagar.
- Listado con tipo, rol, horas, día, boleto perdido y pago.
- Cantidad total de registros, cantidades por tipo y por rol, horas totales y promedio de permanencia.
- Total recaudado, pago mayor y pago menor; mensajes cuando no hay datos.
- Reporte final al salir y mensaje de finalización.

2. ALGORITMO

1. Inicio.

2. Declarar las variables, la matriz de 50 filas y 5 columnas y el arreglo de 50 pagos. Inicializar los contadores y acumuladores en cero.

3. Mostrar las cinco opciones y leer una opción entera válida. Repetir la lectura mientras no esté entre 1 y 5.

4. Seleccionar la acción de acuerdo con la opción.

5. Si la opción es 1 y existen 50 registros, mostrar el mensaje de capacidad alcanzada y continuar en el paso 16.

6. Si la opción es 1 y hay espacio, leer tipo, rol, horas, día y boleto; repetir cada lectura hasta que sea válida.

7. Seleccionar la tarifa por tipo, el factor por día y el descuento por rol según la tabla definida.

8. Calcular $\text{base} = \text{tarifa} \times \text{horas}$ y $\text{pago} = \text{Trunc}((\text{base} \times \text{factor} \times (100 - \text{descuento}) + 5000) / 10000)$. Si el boleto está perdido, sumar 500 centavos.

9. Guardar los cinco datos y el pago en la posición n. Sumar las horas a totalHoras y el pago a total.

10. Si $n = 0$ o $\text{pago} > \text{mayor}$, actualizar mayor. Si $n = 0$ o $\text{pago} < \text{menor}$, actualizar menor. Aumentar n en uno y mostrar la confirmación. Continuar en el paso 16.

11. Si la opción es 2, mostrar las leyendas y recorrer los registros del 0 al $n - 1$ para mostrar sus datos y pagos; si $n = 0$, mostrar que no hay registros. Continuar en el paso 16.

12. Si la opción es 3, mostrar n. Para cada categoría de 1 a 3, poner porTipo y porRol en cero y recorrer todos los registros: aumentar los contadores cuando corresponda. Mostrar ambos conteos por categoría.

13. En la opción 3, mostrar totalHoras y, si $n > 0$, el promedio $\text{totalHoras} / n$; de lo contrario, informar que no está disponible. Continuar en el paso 16.

14. Si la opción es 4, mostrar $\text{total} / 100$; si hay registros, mostrar $\text{mayor} / 100$ y $\text{menor} / 100$; si no, informar que no están disponibles. Continuar en el paso 16.

15. Si la opción es 5, ejecutar las consultas de los pasos 11 a 14 para mostrar el reporte final completo y el mensaje de finalización.

16. Si la opción es diferente de 5, regresar al paso 3; de lo contrario, cerrar el programa.

17. Fin.

El promedio y los valores monetarios se presentan con dos decimales. Los contadores se reinician al consultar estadísticas para evitar que una segunda consulta duplique los resultados.

3. PSEUDOCÓDIGO

Compatible con un perfil de PSeInt que habilite subprocessos, funciones de cadenas e índices en base 0. Copiar todos los subprocessos y el algoritmo principal en un mismo archivo. La matriz utiliza filas 0 a 49 y columnas 0 a 4. PSeInt puede omitir ceros decimales finales al mostrar importes; los valores calculados son los mismos.

SubProceso valor <- LeerEntero(mensaje, minimo, maximo)

Definir mensaje, texto, caracter Como Cadena;
Definir minimo, maximo, valor, k Como Entero;
Definir valido, digito Como Logico;
valido <- Falso;

Mientras No valido Hacer

Escribir Sin Saltar mensaje;

Leer texto;

valor <- 0;

k <- 0;

valido <- Longitud(texto) > 0;

Mientras k < Longitud(texto) Y valido Hacer

caracter <- Subcadena(texto, k, k);

digito <- caracter = "0" O caracter = "1";

digito <- digito O caracter = "2" O caracter = "3";

digito <- digito O caracter = "4" O caracter = "5";

digito <- digito O caracter = "6" O caracter = "7";

digito <- digito O caracter = "8" O caracter = "9";

Si digito Entonces

valor <- valor * 10 + ConvertirANumero(caracter);

Si valor > maximo Entonces

valido <- Falso;

FinSi

SiNo

valido <- Falso;

FinSi

k <- k + 1;

FinMientras

valido <- valido Y valor >= minimo Y valor <= maximo;

Si No valido Entonces

Escribir "Dato invalido. Intente nuevamente.";

FinSi

FinMientras

FinSubProceso

SubProceso MostrarVehiculos(datos Por Referencia, pagos Por Referencia, n)

Definir i, n Como Entero;

Escribir "VEHICULOS REGISTRADOS";

Escribir "Tipos: 1 Moto, 2 Auto, 3 Camioneta.";

Escribir "Roles: 1 Estudiante, 2 Docente, 3 Visitante.";

Escribir "Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si.";

Si n = 0 Entonces

Escribir "No hay vehiculos registrados.";

FinSi

Pseudocódigo — continuación

```
Para i <- 0 Hasta n - 1 Hacer
    Escribir "Registro ", i + 1, " | Tipo ", datos[i,0];
    Escribir "Rol ", datos[i,1], " | Horas ", datos[i,2];
    Escribir "Dia ", datos[i,3], " | Boleto perdido ", datos[i,4];
    Escribir "Pago: $", pagos[i] / 100;
FinPara
FinSubProceso

SubProceso MostrarEstadisticas(datos Por Referencia, n, totalHoras)
    Definir n, totalHoras, categoria, i, porTipo, porRol Como Entero;
    Escribir "ESTADISTICAS";
    Escribir "Vehiculos registrados: ", n;
    Escribir "Tipos: 1 Moto, 2 Auto, 3 Camioneta.";
    Escribir "Roles: 1 Estudiante, 2 Docente, 3 Visitante.";
    Para categoria <- 1 Hasta 3 Hacer
        porTipo <- 0;
        porRol <- 0;
        Para i <- 0 Hasta n - 1 Hacer
            Si datos[i,0] = categoria Entonces
                porTipo <- porTipo + 1;
            FinSi
            Si datos[i,1] = categoria Entonces
                porRol <- porRol + 1;
            FinSi
        FinPara
        Escribir "Tipo ", categoria, ": ", porTipo;
        Escribir "Rol ", categoria, ": ", porRol;
    FinPara
    Escribir "Total de horas: ", totalHoras;
    Si n > 0 Entonces
        Escribir "Promedio: ", Redon(totalHoras / n * 100) / 100;
    SiNo
        Escribir "Promedio de permanencia: no disponible.";
    FinSi
FinSubProceso

SubProceso MostrarRecaudacion(n, total, mayor, menor)
    Definir n, total, mayor, menor Como Entero;
    Escribir "RECAUDACION";
    Escribir "Total recaudado: $", total / 100;
    Si n > 0 Entonces
        Escribir "Mayor pago: $", mayor / 100;
        Escribir "Menor pago: $", menor / 100;
    SiNo
        Escribir "Mayor y menor pago: no disponibles.";
    FinSi
FinSubProceso

Algoritmo ParqueaderoIntegrador
    Definir datos, pagos Como Entero;
    Dimension datos[50,5], pagos[50];
    Definir n, totalHoras, total, mayor, menor Como Entero;
```

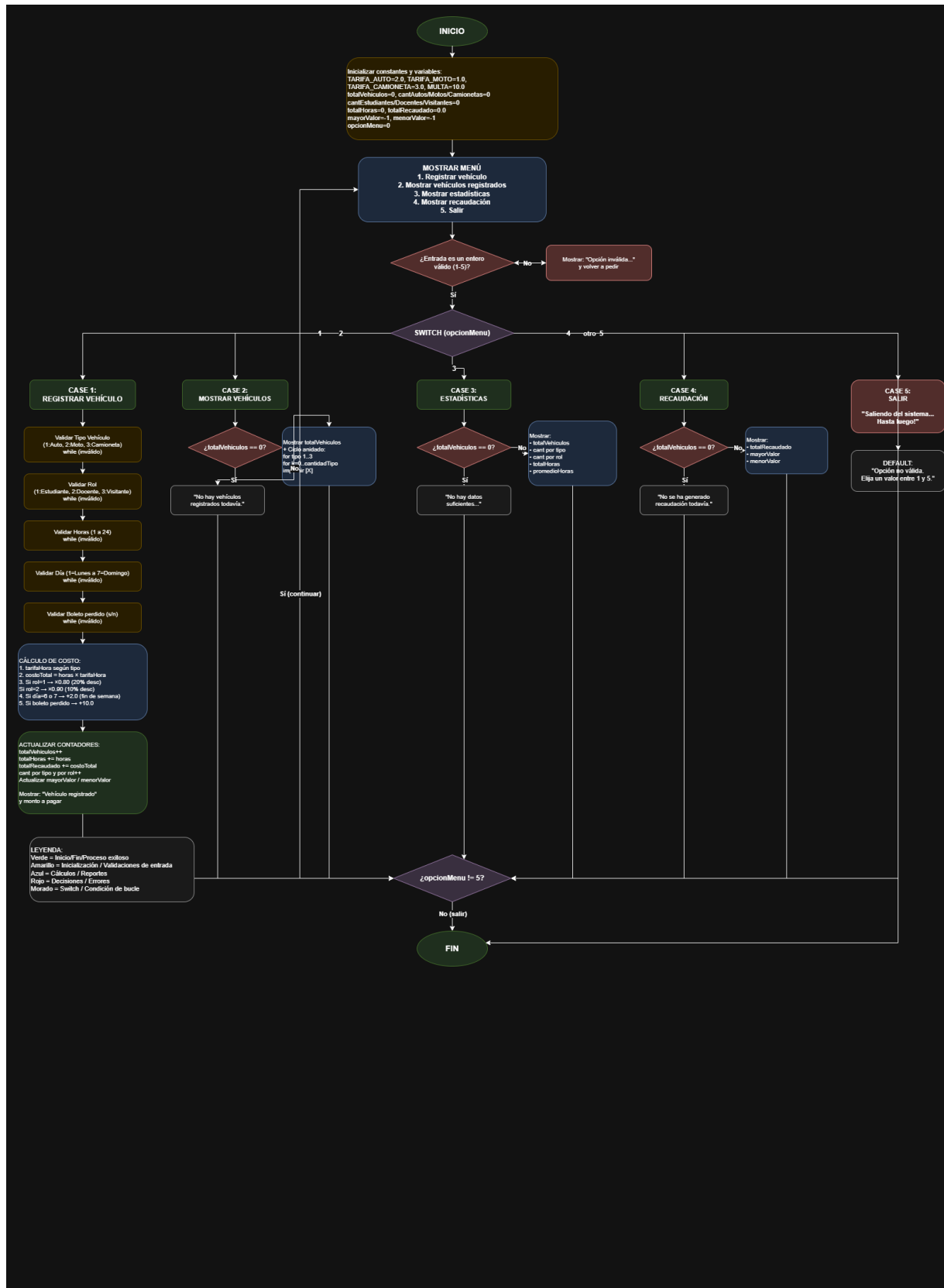
Pseudocódigo — continuación

```
Definir opcion, tipo, rol, horas, dia, boleto Como Entero;
Definir tarifa, factor, descuento, base, pago Como Entero;
n <- 0;
totalHoras <- 0;
total <- 0;
mayor <- 0;
menor <- 0;
Repetir
    Escribir "PARQUEADERO UNIVERSITARIO";
    Escribir "1. Registrar vehiculo";
    Escribir "2. Mostrar vehiculos registrados";
    Escribir "3. Mostrar estadisticas";
    Escribir "4. Mostrar recaudacion";
    Escribir "5. Salir";
    opcion <- LeerEntero("Opcion: ", 1, 5);
    Segun opcion Hacer
        1:
            Si n = 50 Entonces
                Escribir "Capacidad de 50 registros alcanzada.";
            SiNo
                tipo <- LeerEntero("Tipo: 1 Moto, 2 Auto, 3 Camioneta: ", 1, 3);
                rol <- LeerEntero("Rol: 1 Estudiante, 2 Docente, 3 Visitante: ", 1, 3);
                horas <- LeerEntero("Horas (1 a 24): ", 1, 24);
                dia <- LeerEntero("Dia (1 Lunes a 7 Domingo): ", 1, 7);
                boleto <- LeerEntero("Boleto perdido (0 No, 1 Si): ", 0, 1);
                Segun tipo Hacer
                    1: tarifa <- 50;
                    2: tarifa <- 100;
                    3: tarifa <- 150;
                FinSegun
                factor <- 100;
                Si dia = 6 Entonces
                    factor <- 120;
                FinSi
                Si dia = 7 Entonces
                    factor <- 150;
                FinSi
                descuento <- 0;
                Si rol = 1 Entonces
                    descuento <- 20;
                FinSi
                Si rol = 2 Entonces
                    descuento <- 10;
                FinSi
                base <- tarifa * horas;
                pago <- Trunc((base * factor * (100 - descuento) + 5000) / 10000);
                Si boleto = 1 Entonces
                    pago <- pago + 500;
                FinSi
                datos[n,0] <- tipo;
                datos[n,1] <- rol;
                datos[n,2] <- horas;
```

Pseudocódigo — continuación

```
    datos[n,3] <- dia;
    datos[n,4] <- boleto;
    pagos[n] <- pago;
    totalHoras <- totalHoras + horas;
    total <- total + pago;
    Si n = 0 O pago > mayor Entonces
        mayor <- pago;
    FinSi
    Si n = 0 O pago < menor Entonces
        menor <- pago;
    FinSi
    n <- n + 1;
    Escribir "Registro ", n, " guardado. Pago: $", pago / 100;
FinSi
2:
    MostrarVehiculos(datos, pagos, n);
3:
    MostrarEstadisticas(datos, n, totalHoras);
4:
    MostrarRecaudacion(n, total, mayor, menor);
5:
    Escribir "REPORTE FINAL";
    MostrarVehiculos(datos, pagos, n);
    MostrarEstadisticas(datos, n, totalHoras);
    MostrarRecaudacion(n, total, mayor, menor);
    Escribir "Fin del programa.";
FinSegun
Hasta Que opcion = 5
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO



5. PRUEBA DE ESCRITORIO

Caso 1. Sistema vacío y salida inmediata

Se ejecutan las opciones 2, 3, 4 y 5 antes de registrar vehículos. Se verifica que no haya división para cero ni mínimos o máximos inventados.

Variable	Valor
Opciones de entrada	2 → 3 → 4 → 5
n	0
totalHoras / total	0 horas / 0 centavos
porTipo (1, 2, 3)	0, 0, 0
porRol (1, 2, 3)	0, 0, 0
mayor / menor internos	0 / 0; no representan pagos reales
Promedio de permanencia	No disponible
Total recaudado	\$0,00
Mayor y menor pago mostrados	No disponibles
Resultado final	Reporte vacío y finalización normal

Operaciones paso a paso

1. $n = 0$; no se ejecuta el recorrido de registros.
2. Para cada categoría, $\text{porTipo} = 0$ y $\text{porRol} = 0$; el ciclo interno no realiza iteraciones.
3. $\text{totalHoras} = 0$ y $\text{total} = 0$; no se agregan datos.
4. $n > 0$ es falso: no se calcula $0 / 0$.
5. $\text{Total en dólares} = 0 / 100 = \$0,00$. No existe un valor mayor ni menor pagado.
6. Opción 5 presenta el reporte y termina el do-while.

Caso 2. Tres vehículos con tarifas y roles diferentes

Se registran tres vehículos y después se consultan las opciones 2, 3, 4 y 5. Este caso comprueba descuentos, días laborales, sábado, domingo, boleto perdido y acumulación.

Variable	Valor
V1: tipo, rol, horas, día, boleto	2; 1; 3; 1; 0 (auto, estudiante, lunes)
V2: tipo, rol, horas, día, boleto	3; 2; 2; 6; 1 (camioneta, docente, sábado)
V3: tipo, rol, horas, día, boleto	1; 3; 1; 7; 0 (moto, visitante, domingo)
Pagos finales	V1: \$2,40; V2: \$8,24; V3: \$0,75
n / totalHoras	3 / 6
total / mayor / menor	1139 / 824 / 75 centavos
Cantidad por tipo (1, 2, 3)	1, 1, 1
Cantidad por rol (1, 2, 3)	1, 1, 1
Promedio / recaudación	2,00 horas / \$11,39

Registro	Pago (\$)	Horas acum.	Total (\$)	Mayor / menor (\$)
V1	2,40	3	2,40	2,40 / 2,40
V2	8,24	5	10,64	8,24 / 2,40
V3	0,75	6	11,39	8,24 / 0,75

Operaciones paso a paso

V1: base = $\$1,00 \times 3 = \$3,00$. Día: $\$3,00 \times 1,00 = \$3,00$. Descuento: $\$3,00 \times 0,20 = \$0,60$. Pago: $\$3,00 - \$0,60 = \$2,40$. Sin recargo. En centavos: $\text{Trunc}((300 \times 100 \times 80 + 5000) / 10000) = \text{Trunc}(240,5) = 240$.

V2: base = $\$1,50 \times 2 = \$3,00$. Sábado: $\$3,00 \times 1,20 = \$3,60$. Descuento: $\$3,60 \times 0,10 = \$0,36$. Importe: $\$3,60 - \$0,36 = \$3,24$. Boleto perdido: $\$3,24 + \$5,00 = \$8,24$. En centavos: $\text{Trunc}((300 \times 120 \times 90 + 5000) / 10000) + 500 = 324 + 500 = 824$.

V3: base = $\$0,50 \times 1 = \$0,50$. Domingo: $\$0,50 \times 1,50 = \$0,75$. Descuento y recargo: $\$0,00$. En centavos: $\text{Trunc}((50 \times 150 \times 100 + 5000) / 10000) = 75$.

Total de horas = $3 + 2 + 1 = 6$. Promedio = $6 / 3 = 2$ horas. Recaudación = $240 + 824 + 75 = 1139$ centavos = $\$11,39$. Mayor = $\$8,24$; menor = $\$0,75$. Las cantidades por tipo y por rol suman 3 cada una.

Caso 3. Datos inválidos y límite de 24 horas

Se ingresan datos incorrectos antes de los válidos. El registro final corresponde a una motocicleta de visitante durante 24 horas en domingo, con boleto perdido.

Variable	Valor
opcion	9 inválido → 1 válido; luego 5
tipo	4 inválido → 1
rol	0 inválido → 3
horas	0 y 25 inválidos → 24
dia	8 inválido → 7
boleto	2 inválido → 1
Errores de entrada	7 rechazos; no producen registros
tarifa / factor / descuento	50 centavos / 150 % / 0 %
base / pago	1200 / 2300 centavos
n / totalHoras	1 / 24
Cantidad por tipo (1, 2, 3)	1, 0, 0
Cantidad por rol (1, 2, 3)	0, 0, 1
Promedio	24,00 horas
Total / mayor / menor	\$23,00 / \$23,00 / \$23,00

Operaciones paso a paso

1. Los siete intentos inválidos solo generan mensajes y nuevas lecturas; n permanece en 0 hasta completar el registro.
2. Base = $\$0,50 \times 24 = \$12,00$.
3. Domingo = $\$12,00 \times 1,50 = \$18,00$.
4. Visitante: descuento = 0 %; importe = \$18,00.
5. En centavos: $\text{Trunc}((1200 \times 150 \times 100 + 5000) / 10000) = 1800$.
6. Boleto perdido: $1800 + 500 = 2300$ centavos = \$23,00.
7. Primer registro: mayor = menor = 2300; n = 0 + 1 = 1.
8. totalHoras = 0 + 24 = 24; total = 0 + 2300 = 2300.
9. Promedio = $24 / 1 = 24$ horas. El reporte final muestra un solo vehículo.

Resumen de los casos

Caso	Registros	Horas / prom.	Recaudación	Mayor / menor
1. Vacío	0	0 / no disponible	\$0,00	No disponibles
2. Variados	3	6 / 2,00	\$11,39	\$8,24 / \$0,75
3. Validaciones	1	24 / 24,00	\$23,00	\$23,00 / \$23,00

6. CÓDIGO

JAVA

```
import java.util.Scanner;
import java.util.Locale;

public class ParqueroIntegrador {
    // Repetimos la lectura mientras el entero no sea valido.
    static int leerEntero(Scanner sc, String mensaje, int min, int max) {
        String texto;
        double valor;
        boolean valido = false;
        while (!valido) {
            System.out.print(mensaje);
            texto = sc.nextLine();
            if (texto.matches("[0-9]+")) {
                valor = Double.parseDouble(texto);
                if (valor >= min && valor <= max) {
                    return (int) valor;
                }
            }
            System.out.println("Dato invalido. Intente nuevamente.");
        }
        return 0;
    }

    static void mostrarVehiculos(int[][] datos, int[] pagos, int n) {
        int i;
        System.out.println("VEHICULOS REGISTRADOS");
        System.out.println("Tipos: 1 Moto, 2 Auto, 3 Camioneta.");
        System.out.println("Roles: 1 Estudiante, 2 Docente, 3 Visitante.");
        System.out.println("Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si.");
        if (n == 0) {
            System.out.println("No hay vehiculos registrados.");
        }
        for (i = 0; i < n; i++) {
            System.out.printf(Locale.US,
                "Registro %d | Tipo %d | Rol %d | Horas %d%n",
                i + 1, datos[i][0], datos[i][1], datos[i][2]);
            System.out.printf(Locale.US,
                "Dia %d | Boleto perdido %d | Pago: $%.2f%n",
                datos[i][3], datos[i][4], pagos[i] / 100.0);
        }
    }

    static void mostrarEstadisticas(int[][] datos, int n, int totalHoras) {
```

```

int categoria, i, porTipo, porRol;
System.out.println("ESTADISTICAS");
System.out.println("Vehiculos registrados: " + n);
System.out.println("Tipos: 1 Moto, 2 Auto, 3 Camioneta.");
System.out.println("Roles: 1 Estudiante, 2 Docente, 3 Visitante.");
// Ciclos anidados: revisamos los registros de cada categoria.
for (categoria = 1; categoria <= 3; categoria++) {
    porTipo = 0;
    porRol = 0;
    for (i = 0; i < n; i++) {
        if (datos[i][0] == categoria) { porTipo++; }
        if (datos[i][1] == categoria) { porRol++; }
    }
    System.out.println("Tipo " + categoria + ": " + porTipo);
    System.out.println("Rol " + categoria + ": " + porRol);
}
System.out.println("Total de horas: " + totalHoras);
if (n > 0) {
    System.out.printf(Locale.US,
        "Promedio de permanencia: %.2f horas%n", totalHoras / (double) n);
} else {
    System.out.println("Promedio de permanencia: no disponible.");
}
}

static void mostrarRecaudacion(int n, int total, int mayor, int menor) {
    System.out.println("RECAUDACION");
    System.out.printf(Locale.US, "Total recaudado: $%.2f%n", total / 100.0);
    if (n > 0) {
        System.out.printf(Locale.US, "Mayor pago: $%.2f%n", mayor / 100.0);
        System.out.printf(Locale.US, "Menor pago: $%.2f%n", menor / 100.0);
    } else {
        System.out.println("Mayor y menor pago: no disponibles.");
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    final int CAPACIDAD = 50;
    int[][] datos = new int[CAPACIDAD][5];
    int[] pagos = new int[CAPACIDAD];
    int n = 0, totalHoras = 0, total = 0, mayor = 0, menor = 0;
    int opcion, tipo, rol, horas, dia, boleto;
    int tarifa, factor, descuento, base, pago;

    do {
        System.out.println("PARQUEADERO UNIVERSITARIO");
        System.out.println("1. Registrar vehiculo");
        System.out.println("2. Mostrar vehiculos registrados");
        System.out.println("3. Mostrar estadisticas");
        System.out.println("4. Mostrar recaudacion");
        System.out.println("5. Salir");
        opcion = leerEntero(sc, "Opcion: ", 1, 5);
        switch (opcion) {

```

```

case 1:
    if (n == CAPACIDAD) {
        System.out.println("Capacidad de 50 registros alcanzada.");
        break;
    }
    tipo = leerEntero(sc, "Tipo (1 Moto, 2 Auto, 3 Camioneta): ", 1, 3);
    rol = leerEntero(sc, "Rol (1 Estudiante, 2 Docente, 3 Visitante): ", 1, 3);
    horas = leerEntero(sc, "Horas (1 a 24): ", 1, 24);
    dia = leerEntero(sc, "Dia (1 Lunes a 7 Domingo): ", 1, 7);
    boleto = leerEntero(sc, "Boleto perdido (0 No, 1 Si): ", 0, 1);
    tarifa = tipo == 1 ? 50 : (tipo == 2 ? 100 : 150);
    factor = dia <= 5 ? 100 : (dia == 6 ? 120 : 150);
    descuento = rol == 1 ? 20 : (rol == 2 ? 10 : 0);
    base = tarifa * horas;
    // Trabajamos en centavos y redondeamos al centavo mas cercano.
    pago = (base * factor * (100 - descuento) + 5000) / 10000;
    if (boleto == 1) { pago += 500; }
    datos[n][0] = tipo;
    datos[n][1] = rol;
    datos[n][2] = horas;
    datos[n][3] = dia;
    datos[n][4] = boleto;
    pagos[n] = pago;
    totalHoras += horas;
    total += pago;
    if (n == 0 || pago > mayor) { mayor = pago; }
    if (n == 0 || pago < menor) { menor = pago; }
    n++;
    System.out.printf(Locale.US,
        "Registro %d guardado. Pago: $%.2f%n", n, pago / 100.0);
    break;
case 2:
    mostrarVehiculos(datos, pagos, n);
    break;
case 3:
    mostrarEstadisticas(datos, n, totalHoras);
    break;
case 4:
    mostrarRecaudacion(n, total, mayor, menor);
    break;
case 5:
    System.out.println("REPORTE FINAL");
    mostrarVehiculos(datos, pagos, n);
    mostrarEstadisticas(datos, n, totalHoras);
    mostrarRecaudacion(n, total, mayor, menor);
    System.out.println("Fin del programa.");
    break;
    }
} while (opcion != 5);
sc.close();
}
}

```

C++

```
#include <iostream>
#include <iomanip>
#include <string>
#include <sstream>
#include <regex>
using namespace std;

// Repetimos la lectura mientras el entero no sea valido.
int leerEntero(string mensaje, int min, int max) {
    string texto;
    double valor;
    bool valido = false;
    while (!valido) {
        cout << mensaje;
        getline(cin, texto);
        if (regex_match(texto, regex("[0-9]+"))) {
            istringstream lectura(texto);
            if (lectura >> valor) {
                if (valor >= min && valor <= max) {
                    return int(valor);
                }
            }
        }
        cout << "Dato invalido. Intente nuevamente." << endl;
    }
    return 0;
}

void mostrarVehiculos(int datos[][5], int pagos[], int n) {
    int i;
    cout << "VEHICULOS REGISTRADOS" << endl;
    cout << "Tipos: 1 Moto, 2 Auto, 3 Camioneta." << endl;
    cout << "Roles: 1 Estudiante, 2 Docente, 3 Visitante." << endl;
    cout << "Dias: 1 Lunes a 7 Domingo. Boleto: 0 No, 1 Si." << endl;
    if (n == 0) { cout << "No hay vehiculos registrados." << endl; }
    for (i = 0; i < n; i++) {
        cout << "Registro " << i + 1 << " | Tipo " << datos[i][0]
            << " | Rol " << datos[i][1] << " | Horas " << datos[i][2] << endl;
        cout << "Dia " << datos[i][3] << " | Boleto perdido " << datos[i][4]
            << " | Pago: $" << pagos[i] / 100.0 << endl;
    }
}

void mostrarEstadisticas(int datos[][5], int n, int totalHoras) {
    int categoria, i, porTipo, porRol;
    cout << "ESTADISTICAS" << endl;
    cout << "Vehiculos registrados: " << n << endl;
```

```

cout << "Tipos: 1 Moto, 2 Auto, 3 Camioneta." << endl;
cout << "Roles: 1 Estudiante, 2 Docente, 3 Visitante." << endl;
// Ciclos anidados: revisamos los registros de cada categoria.
for (categoria = 1; categoria <= 3; categoria++) {
    porTipo = 0;
    porRol = 0;
    for (i = 0; i < n; i++) {
        if (datos[i][0] == categoria) { porTipo++; }
        if (datos[i][1] == categoria) { porRol++; }
    }
    cout << "Tipo " << categoria << ": " << porTipo << endl;
    cout << "Rol " << categoria << ": " << porRol << endl;
}
cout << "Total de horas: " << totalHoras << endl;
if (n > 0) {
    cout << "Promedio de permanencia: " << totalHoras / double(n)
        << " horas" << endl;
} else {
    cout << "Promedio de permanencia: no disponible." << endl;
}
}

void mostrarRecaudacion(int n, int total, int mayor, int menor) {
    cout << "RECAUDACION" << endl;
    cout << "Total recaudado: $" << total / 100.0 << endl;
    if (n > 0) {
        cout << "Mayor pago: $" << mayor / 100.0 << endl;
        cout << "Menor pago: $" << menor / 100.0 << endl;
    } else {
        cout << "Mayor y menor pago: no disponibles." << endl;
    }
}

int main() {
    const int CAPACIDAD = 50;
    int datos[CAPACIDAD][5], pagos[CAPACIDAD];
    int n = 0, totalHoras = 0, total = 0, mayor = 0, menor = 0;
    int opcion, tipo, rol, horas, dia, boleto;
    int tarifa, factor, descuento, base, pago;
    cout << fixed << setprecision(2);

    do {
        cout << "PARQUEADERO UNIVERSITARIO" << endl;
        cout << "1. Registrar vehiculo" << endl;
        cout << "2. Mostrar vehiculos registrados" << endl;
        cout << "3. Mostrar estadisticas" << endl;
        cout << "4. Mostrar recaudacion" << endl;
        cout << "5. Salir" << endl;
        opcion = leerEntero("Opcion: ", 1, 5);
        switch (opcion) {
            case 1:
                if (n == CAPACIDAD) {
                    cout << "Capacidad de 50 registros alcanzada." << endl;
                    break;
                }

```

```

    }
    tipo = leerEntero("Tipo (1 Moto, 2 Auto, 3 Camioneta): ", 1, 3);
    rol = leerEntero("Rol (1 Estudiante, 2 Docente, 3 Visitante): ", 1, 3);
    horas = leerEntero("Horas (1 a 24): ", 1, 24);
    dia = leerEntero("Dia (1 Lunes a 7 Domingo): ", 1, 7);
    boleto = leerEntero("Boleto perdido (0 No, 1 Si): ", 0, 1);
    tarifa = tipo == 1 ? 50 : (tipo == 2 ? 100 : 150);
    factor = dia <= 5 ? 100 : (dia == 6 ? 120 : 150);
    descuento = rol == 1 ? 20 : (rol == 2 ? 10 : 0);
    base = tarifa * horas;
    // Trabajamos en centavos y redondeamos una sola vez.
    pago = (base * factor * (100 - descuento) + 5000) / 10000;
    if (boleto == 1) { pago += 500; }
    datos[n][0] = tipo;
    datos[n][1] = rol;
    datos[n][2] = horas;
    datos[n][3] = dia;
    datos[n][4] = boleto;
    pagos[n] = pago;
    totalHoras += horas;
    total += pago;
    if (n == 0 || pago > mayor) { mayor = pago; }
    if (n == 0 || pago < menor) { menor = pago; }
    n++;
    cout << "Registro " << n << " guardado. Pago: $"
         << pago / 100.0 << endl;
    break;
case 2:
    mostrarVehiculos(datos, pagos, n);
    break;
case 3:
    mostrarEstadisticas(datos, n, totalHoras);
    break;
case 4:
    mostrarRecaudacion(n, total, mayor, menor);
    break;
case 5:
    cout << "REPORTE FINAL" << endl;
    mostrarVehiculos(datos, pagos, n);
    mostrarEstadisticas(datos, n, totalHoras);
    mostrarRecaudacion(n, total, mayor, menor);
    cout << "Fin del programa." << endl;
    break;
}
} while (opcion != 5);
return 0;
}

```